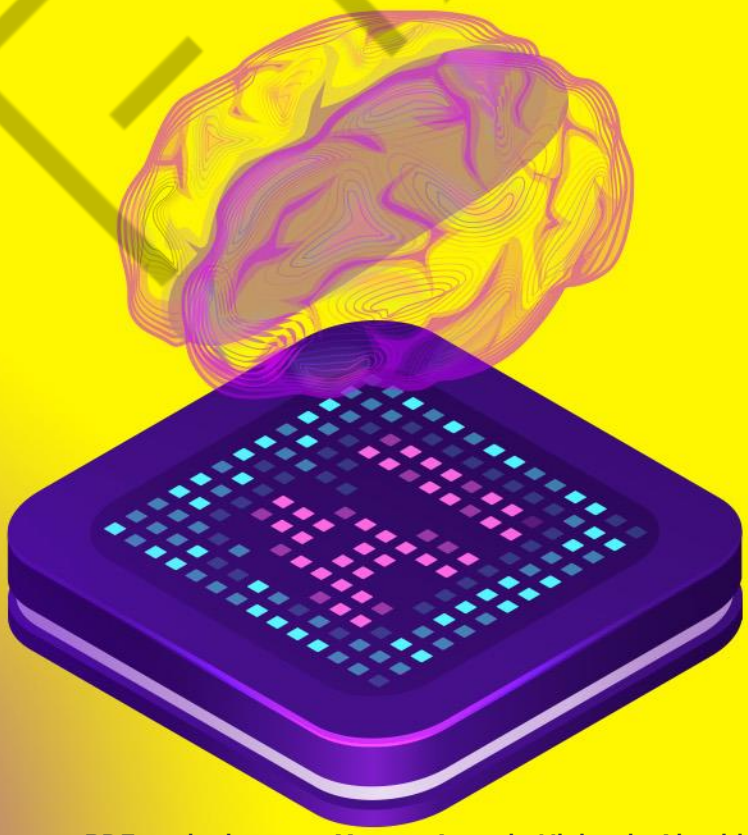
DATA VIZ & DATA SCIENCE

INTRODUÇÃO A INTELIGÊNCIA ARTIFICIAL



PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Principais pesquisadores e marcos da história da Inteligência Artificial...9
Figura 2 – Relação entre os conceitos de Inteligência Artificial, Machine Learning e Deep Learning.....14
Figura 3 – Formas de modelagem 15
Figura 4 – Representação ilustrativa da regressão linear simples 17
Figura 5 – Principais métricas de modelos de regressão.....18
Figura 6 – Representação ilustrativa do KNN 20
Figura 7 – Representação ilustrativa da clusterização 23
Figura 8 – Representação ilustrativa do Método do Cotovelo.....25
Figura 9 – Capa do documentário “Coded Bias” 27
Figura 10 – Tipos de viéses ao trabalhar com IA 28
Figura 11 – Resultado do código-fonte 2 32
Figura 12 – Resultado do código-fonte 3 33
Figura 13 – Resultado do código-fonte 4 33
Figura 14 – Resultado do código-fonte 6 35
Figura 15 – Resultado do código-fonte 7 36
Figura 16 – Resultado do código-fonte 11 38
Figura 17 – Resultado do código-fonte 12 39
Figura 18 – Resultado do código-fonte 16 41
Figura 19 – Resultado do código-fonte 18 42
Figura 20 – Resultado do código-fonte 22 45
Figura 21 – Resultado do código-fonte 23 46
Figura 22 – Resultado do código-fonte 24 46
Figura 23 – Resultado do código-fonte 25 47
Figura 24 – Resultado parcial do código-fonte 26 (alguns histogramas foram omitidos).....48
Figura 25 – Resultado do código-fonte 28 49
Figura 26– Resultado do código-fonte 29 50
Figura 27 – Resultado do código-fonte 30 51
Figura 28 – Resultado do código-fonte 32 53
Figura 29– Resultado do código-fonte 33 54
Figura 30– Resultado do código-fonte 32 adaptado para 5 clusteres 55
Figura 31 – Resultado do código-fonte 36 59
Figura 32 – Resultado do código-fonte 40 61
Figura 33 – Resultado do código-fonte 42 62
Figura 34 – Resultado do código-fonte 42, alterando o ID de cluster para 4 62

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE TABELAS

Tabela 1 – Como construir soluções éticas em IA? 29

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Imports para regressão linear 31
Código-fonte 2 – Leitura do dataset 32
Código-fonte 3 – Informações consolidadas do dataset.....32
Código-fonte 4 – Eliminando dados duplicados 33
Código-fonte 5 – Eliminando colunas inúteis.....34
Código-fonte 6 – Alterando o sistema métrico.....34
Código-fonte 7 – Convertendo a coluna “gender” 35
Código-fonte 8 – Criando a coluna “média”.....36
Código-fonte 9 – Separação de dados em “treino” e “teste” 37
Código-fonte 10 – Criando o modelo de regressão linear simples.....37
Código-fonte 11 – Avaliando o modelo de regressão linear simples.....38
Código-fonte 12 – Plotando os dados de teste e o modelo resultante 38
Código-fonte 13 – Separação de dados em “treino” e “teste” para a regressão múltipla.....40
Código-fonte 14 – Normalização dos dados 40
Código-fonte 15 – Criando o modelo de regressão linear múltipla.....40
Código-fonte 16 – Avaliando o modelo de regressão linear múltipla.....40
Código-fonte 17 – Imports para classificação com KNN 42
Código-fonte 18 – Estruturando os dados como um DataFrame Pandas 42
Código-fonte 19 – Separação de dados em “treino” e “teste” para a classificação ...43
Código-fonte 20 – Padronização e Normalização de dados 43
Código-fonte 21 – Função que aplica e avalia o KNN em função do valor de K 44
Código-fonte 22 – Aplicando “meu_knn()” sobre os dados originais 45
Código-fonte 23 – Aplicando “meu_knn()” sobre os dados padronizados.....45
Código-fonte 24 – Aplicando “meu_knn()” sobre os dados normalizados 46
Código-fonte 25 – Comparação de execuções 46
Código-fonte 26 – Construindo histogramas de cada feature do dataset.....47
Código-fonte 27 – Imports da clusterização com K-Means 49
Código-fonte 28 – Leitura da base “mal.csv”.....49
Código-fonte 29 – Explorando brevemente a base “mal.csv”.....49
Código-fonte 30 – Explorando brevemente a base “mal.csv”.....50
Código-fonte 31 – Aplicando o K-Means.....52
Código-fonte 32 – Gerando visualização dos clusteres formados.....52
Código-fonte 33 – Aplicação do Método do Cotovelo 53
Código-fonte 34 – Proposta de solução para o desafio de regressão – Parte 1/3 57
Código-fonte 35 – Proposta de solução para o desafio de regressão – Parte 2/3 58
Código-fonte 36 – Proposta de solução para o desafio de regressão – Parte 3/3 59
Código-fonte 37 – Proposta de solução para o desafio de classificação – Parte 1/4 59
Código-fonte 38 – Proposta de solução para o desafio de classificação – Parte 2/4 60
Código-fonte 39 – Proposta de solução para o desafio de classificação – Parte 3/4 60
Código-fonte 40 – Proposta de solução para o desafio de classificação – Parte 4/4 61
Código-fonte 41 – Proposta de solução para o desafio de clusterização – Parte 1/2 61
Código-fonte 42 – Proposta de solução para o desafio de clusterização – Parte 2/2 62

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

INTRODUÇÃO À INTELIGÊNCIA ARTIFICIAL 6
1 CONCEITOS DE INTELIGÊNCIA ARTIFICIAL 6
1.1 Inteligência Artificial Geral 7
1.2 Inteligência Artificial Específica 7
2 HISTÓRIA DA INTELIGÊNCIA ARTIFICIAL 8
2.1 Evoluções tecnológicas: do Perceptron até o Deep Learning 10
2.2 Da “Era Dourada”, passando pelo “Inverno de IA”, até os dias futuros 11
3 APLICAÇÕES DE INTELIGÊNCIA ARTIFICIAL 12
4 APRENDIZADO DE MÁQUINA 14
4.1 Formas de modelagem 15
4.2 Regressão linear 16
4.3 Classificação com vizinhos mais próximos 19
4.4 Clusterização de dados 22
5 ÉTICA EM INTELIGÊNCIA ARTIFICIAL: DILEMAS E VIÉSES 26
5.1 Identificando viéses em algoritmos 27
5.2 Construindo soluções éticas com Inteligência Artificial 29
6 DESAFIOS ATUAIS E FUTUROS 30
7 CIÊNCIA DE DADOS EM PYTHON: BIBLIOTECA SKLEARN 30
7.1 Implementando algoritmos supervisionados: regressão 31
7.2 Implementando algoritmos supervisionados: classificação 41
7.3 Implementando algoritmos não-supervisionados: clusterização 48
8 HANDS ON: CONSTRUINDO NOVOS MODELOS DE REGRESSÃO, CLASSIFICAÇÃO E CLUSTERIZAÇÃO 56
8.1 Seu desafio 56
8.2 Proposta de solução 57
REFERÊNCIAS 64
GLOSSÁRIO 65

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

INTRODUÇÃO À INTELIGÊNCIA ARTIFICIAL

1 CONCEITOS DE INTELIGÊNCIA ARTIFICIAL

Olá! Neste capítulo, traremos uma visão ampla sobre Inteligência Artificial, as principais formas de aprendizado e suas aplicações. Também veremos um breve histórico deste campo de pesquisa tão extenso e, atualmente, impossível de ser negligenciado ao pensarmos sobre evolução tecnológica.

Antes de mais nada, vamos buscar um conceito. Pense: o que você entende por “Inteligência Artificial”?

Podemos definir “Inteligência Artificial” de diversas formas. Cada pesquisador adota uma ou outra abordagem, mas uma definição comum seria:

DEFINIÇÃO:

Inteligência Artificial é a simulação de processos da inteligência humana feita por máquinas, especialmente sistemas de computador (software).

Esses processos incluem aprendizado, raciocínio e autocorreção, entre outros.

Pela definição acima, até mesmo a locomoção é um processo que envolve inteligência. Para conseguirmos nos locomover por um cômodo cheio de móveis, precisamos compreender o que há naquele ambiente, traçar uma rota e segui-la desviando de objetos. Em todo momento, nosso cérebro está recebendo informações visuais do ambiente, processamos seu conteúdo e deciframos aquela informação. Por exemplo: este objeto na minha frente é fácil de ser “pulado” ou é grande o suficiente a ponto de eu ter que desviar dele? Assim, a robótica pode ser entendida como um ramo da Inteligência Artificial, pois, ao projetarmos um robô que se desloca de forma autônoma por um ambiente, estamos construindo uma máquina que *simula* um processo que depende de inteligência humana.

Podemos definir a inteligência artificial em duas formas: *inteligência artificial geral* e *inteligência artificial específica*.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.1 Inteligência Artificial Geral

A Inteligência Artificial Geral, também conhecida como "*General AI*", está relacionada com a construção de uma única inteligência artificial capaz de fazer tudo o que o ser humano pode fazer, ou seja, aprender diversas tarefas ao mesmo tempo e se aperfeiçoar.

Em resumo, a Inteligência Artificial Geral vai na linha de "superinteligência", na qual poderíamos ter uma máquina (seja ela hardware e/ou software) capaz de se passar por um ser humano.

Esse tipo de inteligência não é realidade atualmente, sendo encontrada apenas na ficção, em filmes e livros. A inteligência artificial geral é um campo de pesquisa, mas se estamos caminhando com sucesso para esse ponto, é outra história!

1.2 Inteligência Artificial Específica

Já a Inteligência Artificial Específica, também conhecida como "*Narrow AI*", refere-se ao que temos hoje de concreto sobre inteligência artificial: uma série de máquinas (hardware e/ou software) construídas para executar uma única tarefa. Por exemplo: tradução, reconhecimento de voz, busca de conteúdos, predição de doenças, carro autônomo, entre tantas outras aplicações.

As aplicações de inteligência artificial específica estão dominando diversas áreas do conhecimento, e por mais que sua utilização nos passe uma sensação da máquina ser realmente inteligente, ela simplesmente foi treinada/projetada para executar uma única tarefa. Por exemplo, o software do *Google Translator* é bom em tradução de texto, mas não consegue comandar um carro autônomo. Um software de detecção de câncer de pele por meio de imagens de lesões não conseguirá diagnosticar pneumonia em imagens de raio-X de tórax, pois não foi projetado/construído para essa finalidade.

Em resumo, podemos dizer que as inteligências artificiais específicas são soluções inteligentes construídas para um *propósito específico*. Toda vez que nos

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

referimos a uma inteligência artificial, estaremos nos referindo ao conceito de inteligência artificial específica.

2 HISTÓRIA DA INTELIGÊNCIA ARTIFICIAL

Num cenário de grande excitação em torno da Inteligência Artificial (IA), é crucial reconhecer que sua história remonta a várias décadas, marcada por intensa pesquisa e inovação. A história da IA abrangendo aproximadamente oito décadas, testemunhou transformações notáveis desde as suas raízes, na década de 1940. Pioneiros notáveis como Alan Turing, cujo trabalho na década de 1950 delineou as bases conceituais da IA, destacam-se como visionários essenciais.

Entretanto, a jornada da IA não foi isenta de desafios. Na década de 1970, o foco em sistemas especialistas e representação do conhecimento enfrentou críticas e contribuiu para o que ficou conhecido como o "Inverno da IA", um período de desinvestimento e ceticismo. O renascimento, na década de 1990, trouxe a ascensão das redes neurais e o início do aprendizado de máquina, pavimentando o caminho para a explosão de aplicações práticas nas décadas seguintes.

Comparativamente, a história da IA se entrelaça com o avanço geral da computação. Se a história da computação remonta ao início do século XX, com destaque para Alan Turing e a criação da máquina de Turing em 1936, a IA começou a se destacar como um campo autônomo nas décadas seguintes, alcançando o cenário dinâmico e inovador que presenciamos atualmente.

Ao compreender essa trajetória rica e multifacetada, nós ganhamos uma apreciação profunda das raízes e evolução da Inteligência Artificial. Para aprofundar nosso conhecimento, veremos um breve resumo dos principais pesquisadores e marcos da história da Inteligência Artificial, ilustrados na Figura 1.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

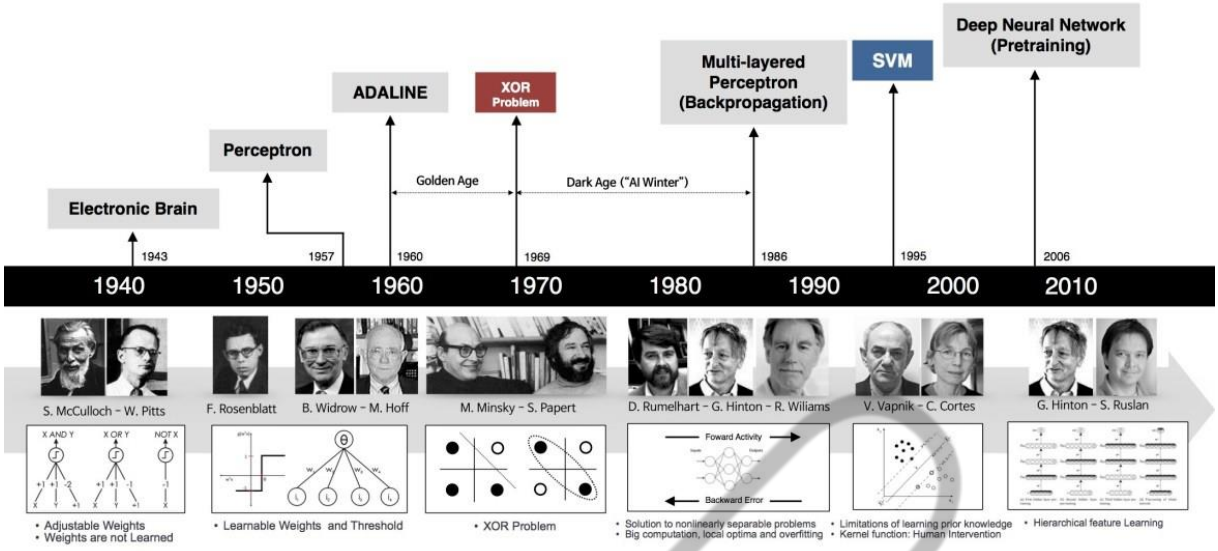


Figura 1 – Principais pesquisadores e marcos da história da Inteligência Artificial
Fonte: Imagem reproduzida de https://i2.wp.com/deeplearningbook.com.br/wp-content/uploads/2017/12/nn_timeline.jpg?w=2133

- **Década de 1940-1950** – *Fundamentos e Visão Inicial*: as origens da IA, com pioneiros como Alan Turing e o conceito de máquinas que poderiam imitar a cognição humana.
- **Década de 1950-1960** – *A Era dos Primeiros Programas de IA*: desenvolvimento de programas que resolviam problemas específicos, como o "Logic Theorist", de Newell e Simon.
- **Década de 1960-1970** – *O Período de Otimismo e Desafios*: crescimento da IA com o surgimento da linguagem de programação Lisp e otimismo inicial, seguido por desafios e críticas.
- **Década de 1970-1980** – *A Revolução Conhecimento e Sistemas Periciais*: ênfase em sistemas especialistas e representação do conhecimento para resolver problemas específicos.
- **Década de 1980-1990** – *Do Inverno da IA à Ascensão da Conexão*: o "Inverno da IA", com redução de financiamento, seguido pelo renascimento com foco em redes neurais e aprendizado de máquina.
- **Década de 1990-2000** – *A Revolução da Internet e Aprendizado Profundo*: a Internet impulsiona a coleta de dados, e o início do século

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

XXI testemunha a ascensão do aprendizado profundo, marcando a era contemporânea da IA.

- **Década de 2010-2020** – *Aplicações Práticas e Ética*: explosão de aplicações práticas em IA, mas também crescente atenção à ética, privacidade e impactos sociais.
- **Década de 2020 em diante** – *Perspectivas Futuras*: desenvolvimentos contínuos em IA, incluindo aprimoramentos na compreensão natural da linguagem, visão computacional, IA explicável e abordagens éticas e responsáveis.

2.1 Evoluções tecnológicas: do Perceptron até o Deep Learning

A primeira implementação de uma rede neural funcional data de 1957, com o desenvolvimento do Perceptron, proposto por Frank Rosenblatt. O Perceptron foi um marco ao introduzir o conceito de uma unidade de processamento que podia aprender e tomar decisões. Apesar das limitações no tratamento de problemas complexos, sua arquitetura semeou as bases para redes neurais.

As aplicações práticas para o Perceptron passaram a surgir após sua proposição, culminando no ADALINE (*Adaptive Linear Neuron*) de Bernard Widrow e Marcian Hoff, de 1960. O ADALINE foi uma extensão do Perceptron, introduzindo a capacidade de aprendizado adaptativo. Suas principais inovações estão no ajuste gradativo dos pesos de forma contínua, bem como em sua função de ativação linear, mas com saídas contínuas (ao contrário das saídas binárias do Perceptron).

O ADALINE foi um passo crucial na evolução dos modelos lineares de aprendizado. Graças a sua capacidade de aprender padrões, o ADALINE ainda é utilizado em áreas como processamento de sinais e comunicação, para filtragem adaptativa e equalização de canais.

Já em 1969, Marvin Minsky e Seymour Papert, em seu livro "Perceptrons", destacaram as limitações do Perceptron ao abordar problemas não linearmente separáveis, como o “XOR Problem”. Isso influenciou uma fase de ceticismo e desinvestimento na IA, conhecida como o "Inverno da IA".

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Apenas em 1986, a “propagação do erro retroativo” (*backpropagation*), desenvolvida por David Rumelhart, Geoffrey Hinton e Ronald Williams, permitiu treinar redes neurais com múltiplas camadas. Isso superou as limitações anteriores, levando a um renascimento do interesse em redes neurais e inaugurando a pesquisa em redes neurais multicamadas.

A evolução das redes neurais multicamadas culminou no surgimento de redes reunais cada vez mais profundas, termo conhecido como “Deep Learning”. De 2012 em diante, o Deep Learning tem transformado o cenário da IA, com as redes convolucionais (CNNs) e as redes recorrentes (RNNs). Nomes como Geoffrey Hinton, Yann LeCun e Yoshua Bengio desempenharam papéis cruciais, tornando o Deep Learning uma força dominante em aplicações modernas de IA.

2.2 Da “Era Dourada”, passando pelo “Inverno de IA”, até os dias futuros

A história da IA está intimamente ligada com a história das redes neurais. Desde a primeira grande época de empolgação com os resultados práticos (era dourada, após Perceptron e ADALINE), esse campo de pesquisa passou por momentos sombrios, conhecido como o “Inverno de IA”.

O “Inverno de IA” é o período caracterizado pelo insucesso das pesquisas, avanços insignificantes e falta de investimentos. Vale lembrar que muita da limitação para se atingir resultados relevantes era devido à falta de dados e, sobretudo, de falta de poder computacional para construção e treinamento de modelos complexos.

O Deep Learning foi uma revolução, pois surgiu no momento propício: na década de 2010, já contávamos com a Internet, a geração e compartilhamento de grandes massas de dados, bem como poder computacional relevante para treinar redes neurais profundas.

Desde o início de 2023, estamos vivendo outra “grande era dourada” de IA, impulsionada pelos resultados fantásticos das IAs Generativas, que não só aprendem a reconhecer padrões, mas aplicá-los para gerar novos dados, com percepção altamente realista. Hoje em dia, a IA é capaz de, por meio de Deep Learning, gerar imagens, vídeos, áudios e textos com qualidade surpreendente. Cabe agora

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

identificarmos nichos de aplicação frutíferos para esta tecnologia, bem como direcionar sua aplicabilidade pautada na ética.

3 APLICAÇÕES DE INTELIGÊNCIA ARTIFICIAL

Existem diversas aplicabilidades para Inteligência Artificial. Cada autor ou pesquisador propõe uma divisão para as subáreas. Certamente há muita intersecção entre elas. Por exemplo, podemos fazer uma divisão de aplicações de acordo com o tipo de dado. Nesse caso, textos seriam encaixados sob o guarda-chuva de Processamento de Linguagem Natural (PLN, do Inglês *Natural Language Processing* – NLP). Mas é possível trabalhar com Machine Learning usando texto. Inclusive, as melhores soluções de Machine Learning para texto são atualmente feitas por meio de Deep Learning.

Sem pretensão de sermos exaustivos nas aplicações de Inteligência Artificial, podemos dividi-las grosseiramente nas seguintes categorias:

- **Raciocínio Baseado em Conhecimento (*Knowledge-Based Systems*)**: aqui nos referimos à aplicação de padrões e processamento de regras pré-definidas.
    - Aplicações: sistemas especialistas, diagnóstico médico, planejamento automatizado.
- **Algoritmos Bio-inspirados**: técnicas de inteligência artificial inspiradas em processos biológicos.
    - Aplicações: aprendizado de máquina evolutivo, algoritmos genéticos, redes neurais artificiais.
- **Processamento de Linguagem Natural**: processamento de áudio e textos.
    - Aplicações: Tradução automática, chatbots, análise de sentimentos, sumarização de texto, questionamento e resposta automática.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Visão Computacional**: processamento de imagens e vídeos.
    - Aplicações: reconhecimento de objetos, detecção facial, análise de vídeo, reconhecimento de caracteres em imagens.
- **Robótica**: automação em nível de hardware.
    - Aplicações: robôs autônomos, automação industrial, assistência em saúde, exploração espacial.
- **Predição de séries temporais**: quando os dados possuem uma dependência temporal, tal dependência deve ser respeitada e explorada pelos algoritmos. Essa é uma área de atuação forte das Redes Neurais Recorrentes.
    - Aplicações: prever valor de venda de ações, estimar quantidades de produtos em estoques, avaliar arritmias em eletrocardiogramas.
- **Aprendizado por Reforço (*Reinforcement Learnin*g)**: aprendizado on-line influenciado por regras pré-estabelecidas de atuação de um agente em um ambiente.
    - Aplicações: jogos, robótica autônoma, otimização de recursos.
- **Aprendizado de Máquina (*Machine Learning*)**: máquinas aprendendo a tomar decisões de forma a partir da identificação de padrões em dados.
    - Aplicações: classificação de documentos, reconhecimento de padrões, sistemas de recomendação, reconhecimento de voz.

Nessa disciplina entenderemos como as redes neurais artificiais são criadas e treinadas, bem como veremos sua evolução para o Deep Learning. Apesar de as redes neurais artificiais serem um tipo de algoritmo bioinspirado, não deixam de ser um dos diversos algoritmos de aprendizado de máquina. Inclusive, Deep Learning (que são essencialmente redes neurais artificiais profundas) são as técnicas mais promissoras para aplicações com sucesso de Visão Computacional e Processamento de Linguagem Natural (Figura 2). Viu como tudo está interligado?

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

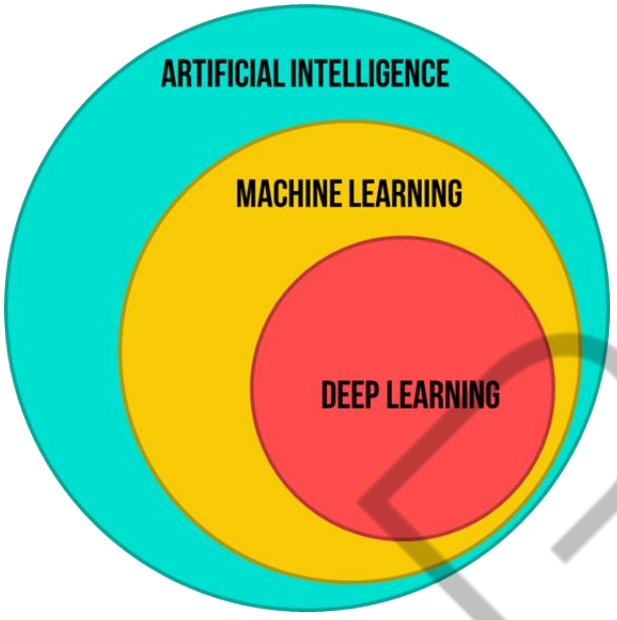


Figura 2 – Relação entre os conceitos de Inteligência Artificial, Machine Learning e Deep Learning
Fonte: Imagem reproduzida de https://www.advancinganalytics.co.uk/blog/2021/12/15/understanding-the-difference-between-ai-ml-and-dl-using-an-incredibly-simple-example

Devido à natureza intrínseca dos conceitos, vamos retomar as definições de aprendizado de máquina e vermos exemplos típicos de algoritmos de classificação, regressão e clusterização. Adiante, esses conceitos serão revisitados à luz de redes neurais artificias e Deep Learning.

4 APRENDIZADO DE MÁQUINA

Vimos que o Aprendizado de Máquina é uma área da Inteligência Artificial que visa tornar as máquinas inteligentes por meio de dados. Os dados são exemplos que as máquinas utilizam para detectar padrões e tendências e, a partir destes, traçar respostas para novos dados que são apresentados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

4.1 Formas de modelagem

Dentro de aprendizado de máquina existem 3 formas de modelagem de dados (Figura 3):

Figura 3 – Formas de modelagem
Fonte: Autor

- **Aprendizado Supervisionado (*Supervised Learning*)**: nesse método, o modelo é treinado em um conjunto de dados rotulado (seus *labels*), onde cada exemplo de entrada está associado a uma saída conhecida.
    - *Objetivo*: o modelo aprende a mapear relações entre entradas e saídas, permitindo prever saídas para novas entradas não vistas durante o treinamento.
    - *Exemplos de Aplicações*: classificação (identificar a categoria de uma entrada), Regressão (prever um valor contínuo).
- **Aprendizado Não Supervisionado (*Unsupervised Learning*)**: nesse caso, o modelo é treinado em um conjunto de dados não rotulado, onde o algoritmo busca identificar padrões, estruturas ou relações intrínsecas nos dados.
    - *Objetivo*: o modelo aprende a estrutura subjacente dos dados sem a orientação explícita de rótulos, explorando características e agrupamentos naturais.
    - *Exemplos de Aplicações*: agrupamento (organizar dados em grupos), Redução de Dimensionalidade.
- **Aprendizado por Reforço (*Reinforcement Learning*)**: nesse paradigma, o modelo, chamado agente, interage com um ambiente

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

dinâmico e recebe feedback na forma de recompensas ou penalidades com base nas ações que realiza.

- *Objetivo*: o agente aprende a realizar ações que maximizam a recompensa ao longo do tempo, ajustando seu comportamento com base nas interações com o ambiente.
- *Exemplos de Aplicações*: Jogos (como xadrez ou Go), Robótica Autônoma, Sistemas de Controle.

Nesse primeiro momento recapitularemos os algoritmos fundamentais de regressão linear, classificação e clusterização. Aproveitaremos para retomar os principais conceitos, introduzir aspectos práticos de cada algoritmo e convidá-los a construir seus primeiros modelos.

Adiante, veremos como implementar cada uma dessas estratégias (e também aprendizado por reforço) por meio de redes neurais artificiais. Vamos em frente?

4.2 Regressão linear

A Regressão Linear é um método estatístico que busca modelar a relação linear entre uma variável dependente (alvo) e uma ou mais variáveis independentes (preditoras). É uma ferramenta fundamental em Aprendizado de Máquina e Estatística, usada para entender e prever a relação entre variáveis. Veja o exemplo hipotético da Figura 4.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

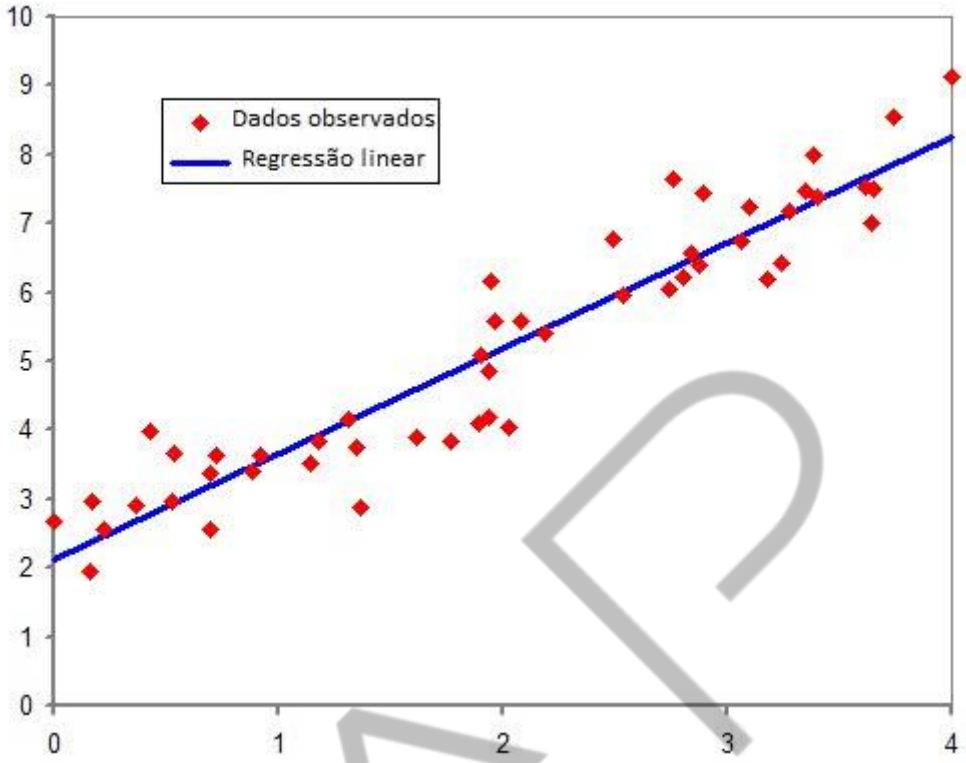


Figura 4 – Representação ilustrativa da regressão linear simples
Fonte: Imagem reproduzida de https://www.dicionariofinanceiro.com/regressao-linear/

A Figura 4 mostra a regressão linear mais simples possível, quando UMA variável independente (X) modela uma variável dependente (Y). Ao plotar os pontos reais (exemplos de treinamento), que são os pontos em vermelho, podemos notar uma tendência de dispersão (conforme X aumenta, Y aumenta também). Essa relação pode ser aproximada por uma reta, que é o nosso modelo (a reta em azul).

Essa relação linear é perfeita? Claramente não, mas é uma aproximação. Qualquer modelo preditivo, seja baseado em Deep Learning ou não, é uma aproximação de um fenômeno sendo estudado.

O modelo em azul pode ser expresso como uma reta, na forma “y = a*x + b”. Assim, dado um novo exemplo “x”, descobrimos o valor predito “y”, que assume um valor contínuo. O pulo do gato é: como descobrir os coeficientes “a” e “b”?

A descoberta dos coeficientes é justamente o treinamento do modelo. O método dos Mínimos Quadrados é frequentemente usado para ajustar a linha aos

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

dados. Ele minimiza a soma dos quadrados das diferenças entre os valores observados e os valores previstos pela linha de regressão.

Esse modelo pode ser facilmente estendido: caso tenhamos outras variáveis independentes (X1, X2, X3, ..., Xn), faríamos uma regressão múltipla. O processo de treinamento e validação é o mesmo, como veremos na parte prática.

Potenciais aplicações para essa forma de modelagem incluem:

- **Predições**. Exemplo: prever o desempenho de um aluno com base no número de horas estudadas;
- **Análise de Dados Temporais**. Exemplo: prever vendas mensais com base em dados históricos;
- **Otimização de Recursos**. Exemplo: estimar o tempo de execução de um algoritmo em função do tamanho da entrada.

<u>Como avaliar métodos de regressão?</u>

As métricas de modelos de regressão são ferramentas essenciais para avaliar o desempenho e a precisão de modelos que tentam prever valores contínuos. A Figura 5 ilustra algumas dessas métricas-chave:

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}|$$

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y})^2$$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y})^2}$$

$$R^2 = 1 - \frac{\sum(y_i - \hat{y})^2}{\sum(y_i - \bar{y})^2}$$

Figura 5 – Principais métricas de modelos de regressão
Fonte: Imagem reproduzida de https://www.datatechnotes.com/2019/02/regression-model-accuracy-mae-mse-rmse.html#google_vignette

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Na Figura 5, é importante notar que “y” se refere ao valor real daquela observação (label coletado), “Ŷ” é o valor predito de “y” e “ȳ” é a média dos valores reais de “y”.

Cada métrica possui um nome e significado, bem como uma forma para interpretá-la, como se segue:

- **Erro Médio Absoluto (MAE – *Mean Absolute Error*)**: representa a média das diferenças absolutas entre as previsões e os valores reais. É robusto a outliers, mas não penaliza grandes erros de maneira significativa;
- **Erro Quadrático Médio (MSE – *Mean Squared Error*)**: calcula a média dos quadrados das diferenças entre previsões e valores reais. Penaliza erros maiores mais fortemente do que o MAE;
- **Raiz do Erro Quadrático Médio (RMSE – *Root Mean Squared Error*)**: semelhante ao MSE, mas a raiz quadrada é aplicada para retornar a métrica à escala original dos dados. Fornece uma interpretação mais intuitiva dos erros;
- **Coeficiente de Determinação (R^2)**: mede a proporção da variabilidade na variável dependente que é explicada pelo modelo. Varia de 0 a 1, sendo 1 indicativo de um ajuste perfeito.

Com exceção do R^2, que possui uma variação de 0 a 1, as demais métricas variam de 0 ao infinito, e quanto menores, melhor! Afinal, se elas medem o ERRO, quanto menor o erro do modelo, melhor o seu ajuste.

Praticaremos a construção e avaliação de modelos preditivos de regressão no final do capítulo.

4.3 Classificação com vizinhos mais próximos

O algoritmo dos vizinhos mais próximos, conhecido como “KNN” (do Inglês “*K-Nearest Neighbors*”) é um algoritmo de aprendizado supervisionado usado para

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

classificação e regressão. Aqui, estamos focados na aplicação de classificação. Ele fundamenta-se no princípio de que exemplos semelhantes devem ter rótulos (*labels*, classes) semelhantes ou valores semelhantes.

Considere o seguinte exemplo hipotético: de acordo com o nível de utilização de 2 substâncias (X e Y), a combinação resultaria em um *anticoagulante* ou um *antibiótico*. A distribuição dos exemplos de tais combinações está na Figura 6.

Figura 6 – Representação ilustrativa do KNN
Fonte: Imagem reproduzida das notas de aula do Prof. Hellynson

Ao plotar nesse espaço uma nova amostra, com determinado valor de X ou Y, cujo rótulo não é conhecido, podemos verificar o rótulo de K vizinhos mais próximos para predizer o rótulo na nova amostra.

Assim, em termos de funcionamento, o KNN armazena os dados de treinamento, que consistem em instâncias com suas características e rótulos de classe associados.

Quando uma nova instância de teste é apresentada, o KNN calcula a distância entre essa instância e todas as instâncias de treinamento no espaço de características. A distância euclidiana é comumente usada, mas outras métricas também podem ser aplicadas, dependendo do problema.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O algoritmo seleciona os K vizinhos mais próximos à nova instância com base nas distâncias calculadas. Esse valor de K é uma escolha crucial e afeta a sensibilidade do modelo.

No caso da classificação, o KNN atribui à nova instância a classe mais frequente entre seus K vizinhos mais próximos. Alternativamente, pode-se ponderar os votos dos vizinhos, dando mais peso aos vizinhos mais próximos. A nova instância é atribuída à classe que recebeu a maioria dos votos dos vizinhos mais próximos.

IMPORTANTE: o KNN é um algoritmo simples e intuitivo, mas a escolha adequada do K é crucial. Valores pequenos podem tornar o modelo sensível a outliers, enquanto valores grandes podem suavizar a fronteira de decisão. É importante também normalizar as características para evitar que aquelas com escalas maiores dominem o cálculo da distância.

Potenciais aplicações para modelos de classificação incluem:

- **Detecção de Spam**: classificação de e-mails como spam ou legítimos, com base em padrões de texto e comportamento;
- **Diagnóstico Médico**: identificação de doenças a partir de dados médicos, como exames laboratoriais e imagens;
- **Análise de Sentimento**: avaliação automatizada de sentimentos em textos, como em redes sociais ou avaliações de produtos;
- **Previsão de Churn**: antecipação da possibilidade de um cliente deixar de usar um serviço ou produto;
- **Detecção de Fraudes Financeiras**: identificação de transações suspeitas com base em padrões de comportamento.

Entre tantos outros que veremos adiante!

Como avaliar métodos de classificação?

As métricas de modelos de classificação são ferramentas essenciais para avaliar seu desempenho. Existe uma infinidade de métricas, mas aqui vamos listar as principais:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Acurácia (*Accuracy*)**: proporção de instâncias classificadas corretamente em relação ao total de instâncias;
- **Sensibilidade (*Sensibility*)**: proporção de instâncias classificadas como verdadeiramente positivas em relação ao total de instâncias verdadeiramente positivas;
- **Especificidade (*Specificity*)**: proporção de instâncias classificadas como verdadeiramente negativas em relação ao total de instâncias verdadeiramente negativas;
- **Área sob a Curva ROC (*AUC-ROC*)**: mede a capacidade do modelo de distinguir entre classes, traçando a curva ROC e calculando a área sob ela;
- **Matriz de Confusão**: uma visão tabular das previsões do modelo, destacando verdadeiros positivos, verdadeiros negativos, falsos positivos e falsos negativos.

Praticaremos a construção e avaliação de modelos preditivos de classificação no final do capítulo.

4.4 Clusterização de dados

A clusterização de dados é uma técnica de aprendizado não supervisionado que visa agrupar instâncias de dados similares em conjuntos distintos, chamados de clusters. O objetivo é que instâncias dentro de um mesmo cluster sejam mais semelhantes entre si do que com instâncias de outros clusters. Esse processo pode ser entendido pela Figura 7:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

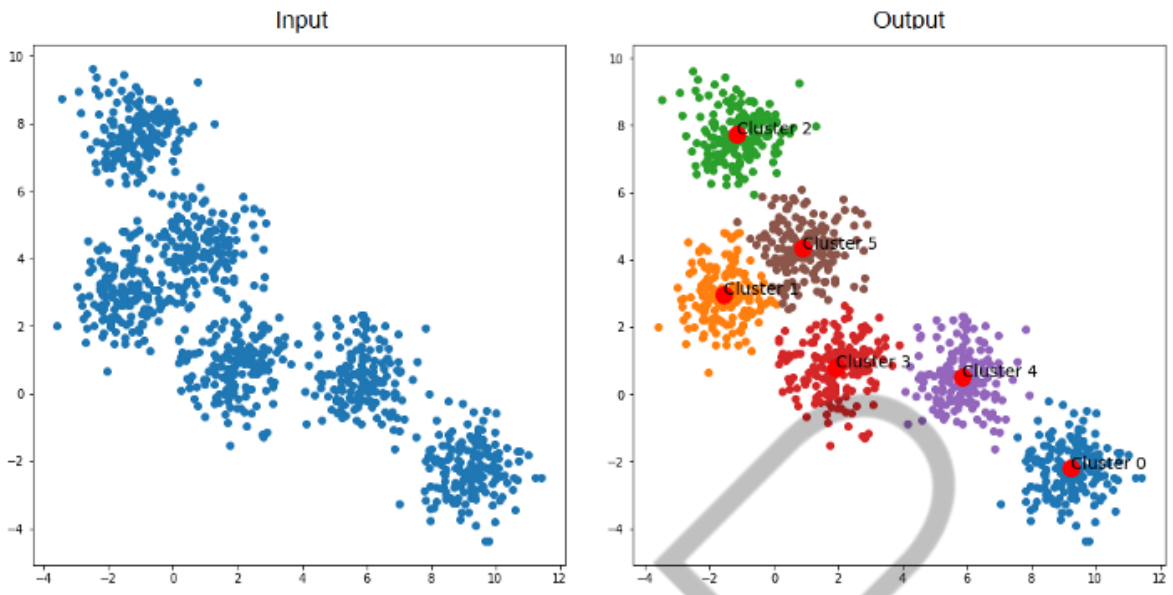


Figura 7 – Representação ilustrativa da clusterização
Fonte: Imagem reproduzida de https://blog.somostera.com/data-science/clusteriza%C3%A7%C3%A3o-de-dados

A Figura 7 mostra um conjunto de pontos em azul, entendidos sob duas dimensões (X e Y). Nós não temos os labels desses exemplos, nem uma pista sobre seus significados. No entanto, ao analisar o “Input”, podemos notar a tendência de grupos, os quais são reforçados no “Output”.

Assim como métodos de regressão e classificação, existem diversos algoritmos de clusterização. O mais comum é o **K-Means (K-Médias)**. O algoritmo K-Means funciona da seguinte maneira:

1. **Inicialização dos Centróides**: o algoritmo começa selecionando aleatoriamente K pontos de dados como os centróides iniciais dos clusters;
2. **Atribuição aos Clusters**: cada ponto de dados é atribuído ao cluster cujo centróide é o mais próximo, usando alguma medida de distância, frequentemente a distância euclidiana;
3. **Atualização dos Centróides**: os centróides dos clusters são recalculados como as médias dos pontos atribuídos a cada cluster;
4. **Repetição**: os passos 2 e 3 são repetidos iterativamente até que não haja mais mudanças significativas na atribuição dos pontos aos clusters ou até que um número máximo de iterações seja atingido.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

IMPORTANTE: devido à natureza espacial da clusterização, é de suma importância normalizar/padronizar as features antes de aplicar o algoritmo de clusterização K-Means.

Voltando à Figura 7, você pode discordar da existência de 6 grupos distintos, alegando que os grupos 1 (laranja) e 5 (marrom) deveriam ser unidos gerando 5 grupos finais. Pois é, a clusterização é uma forma subjetiva de trabalhar com Machine Learning devido à sua dependência da interpretação humana na definição de significado e relevância nos agrupamentos identificados.

A subjetividade emerge em várias fases do processo, desde a escolha do algoritmo e número de clusters até a interpretação final dos resultados. A definição do número ótimo de clusters muitas vezes carece de uma abordagem objetiva clara, exigindo a aplicação de métodos heurísticos, como o Método do Cotovelo, que ainda dependem da intuição do analista. Além disso, a interpretação dos clusters formados é altamente subjetiva, já que a significância dos agrupamentos depende do domínio do problema e dos objetivos específicos da análise.

Como descobrir o "K" ideal para o K-Means?

O Método do Cotovelo é uma abordagem heurística usada na análise de clusters para determinar o número ótimo de clusters em algoritmos como o K-Means. Esse método envolve calcular a inércia (soma das distâncias quadráticas dos pontos aos centróides) para diferentes valores de K e identificar o ponto onde a diminuição da inércia começa a desacelerar, formando um padrão que se assemelha a um cotovelo.

A Figura 8 ilustra esse processo: basicamente, aplicamos o K-Means sobre os dados a partir de diferentes valores de K. O ponto de maior inflexão da curva é o melhor valor de K possível para aquele conjunto de dados. No exemplo abaixo, o valor selecionado é K=3.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

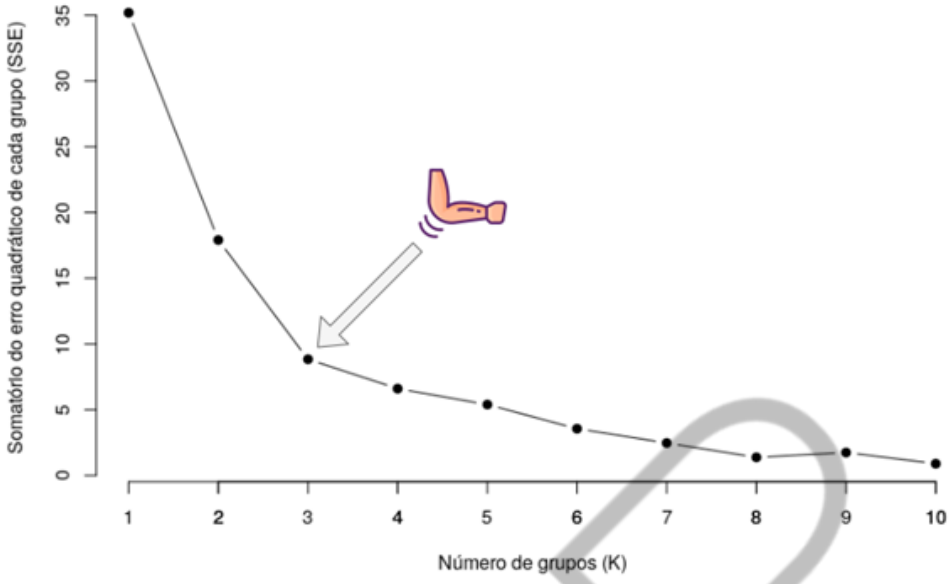


Figura 8 – Representação ilustrativa do Método do Cotovelo
Fonte: Imagem reproduzida de https://dataat.github.io/introducao-ao-machine-learning/agrupamento.html

O ponto no qual a curva exibe uma mudança mais acentuada, indicado pelo cotovelo, é considerado uma estimativa razoável para o número de clusters ideais. Contudo, é importante destacar que a escolha de K utilizando o Método do Cotovelo é subjetiva e depende, em última análise, da interpretação do analista.

IMPORTANTE: em casos onde o cotovelo não é claramente definido, outros métodos e a intuição do especialista podem ser necessários para uma decisão mais precisa sobre o número de clusters.

Potenciais aplicações para modelos de clusterização incluem:

- **Segmentação de Mercado**: agrupamento de clientes com base em comportamentos de compra para estratégias de marketing direcionadas;
- **Recomendação Personalizada**: agrupamento de usuários com preferências similares para recomendações de produtos ou conteúdo mais relevantes;
- **Detecção de Anomalias**: identificação de comportamentos fora do padrão em sistemas, indicando possíveis fraudes ou falhas;

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Categorização de Documentos**: agrupamento de textos para organização eficiente, como em sistemas de gerenciamento de documentos;
- **Organização de Redes Sociais**: agrupamento de perfis de usuários para facilitar a descoberta de conexões e interesses similares.

Entre várias outras aplicações!

Praticaremos a clusterização no final do capítulo.

5 ÉTICA EM INTELIGÊNCIA ARTIFICIAL: DILEMAS E VIÉSES

A rápida evolução da Inteligência Artificial (IA) traz consigo não apenas promessas inovadoras, mas também uma série de desafios éticos que requerem atenção cuidadosa. A ética em IA abrange uma variedade de princípios e preocupações, incluindo transparência, equidade, responsabilidade e privacidade.

Esse tema está pautado em dois conceitos fundamentais:

- **Ética em IA**: refere-se à aplicação de princípios éticos no desenvolvimento, implementação e uso de sistemas de IA. Envolve considerar as implicações éticas de algoritmos autônomos, aprendizado de máquina e automação em diversos setores. Dilemas éticos emergem quando as decisões de máquinas impactam a vida das pessoas, influenciam processos decisórios ou perpetuam desigualdades;
- **Dilemas éticos**: um dilema ético central é a tomada de decisões automatizada. Por exemplo, em sistemas de justiça criminal, algoritmos de previsão de reincidência podem introduzir preconceitos e reforçar desigualdades sociais. O equilíbrio entre automatizar decisões e garantir justiça é um dilema persistente. Outros dilemas incluem privacidade em sistemas de vigilância por câmeras com reconhecimento facial e a questão da responsabilidade quando algoritmos autônomos tomam decisões cruciais.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A Figura 9 ilustra a capa do documentário “*Coded Bias*”, que mostra de forma didática os conceitos e perigos dos viéses codificados nos algoritmos de IA. "Coded Bias" aborda a pesquisadora Joy Buolamwini, que descobre viéses em algoritmos de reconhecimento facial que discriminam pessoas de pele mais escura e mulheres. O documentário destaca as implicações éticas desses viéses tecnológicos e aponta para a necessidade de regulamentação e responsabilidade na IA para evitar discriminações sistêmicas.

Figura 9 – Capa do documentário “Coded Bias”
Fonte: Imagem reproduzida de https://cinepop.com.br/critica-coded-bias-netflix-lanca-incrivel-documentario-aos-moldes-de-o-dilema-das-redes-290123/

5.1 Identificando viéses em algoritmos

Existem diversas fontes de viéses ao se trabalhar com algoritmos de IA, como mostra a Figura 10:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

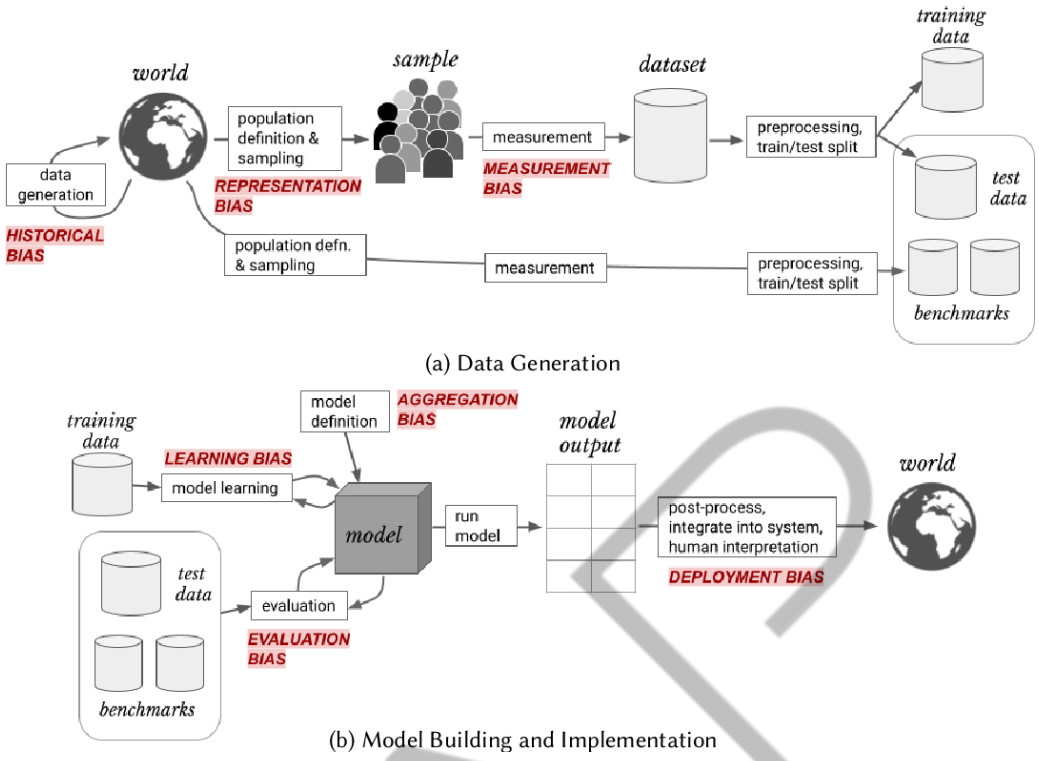


Figura 10 – Tipos de viéses ao trabalhar com IA
Fonte: Imagem reproduzida de https://cinepop.com.br/critica-coded-bias-netflix-lanca-incrivel-documentario-aos-moldes-de-o-dilema-das-redes-290123/

Algoritmos de IA podem herdar e amplificar **viéses presentes nos dados** de treinamento. Um exemplo notório foi a discriminação racial em sistemas de pontuação de crédito que resultaram em ofertas desiguais. Os viéses dos dados podem ser uma consequência da seleção dos exemplos que compõem aquela base de dados, ou a geração do dado que perpetua viéses históricos.

Do ponto de vista de Machine Learning, podemos também ter **viéses de mensuração** ao aplicar métricas que não fazem boas medidas dos erros/acertos do modelo preditivo. Por exemplo, imagine que num conjunto de testes com 100 indivíduos, 80 estejam sadios e 20 estejam doentes; um modelo preditivo que apenas chuta que todos estão sadios teria uma acurácia de 80%, o que parece bom, mas na verdade, não demonstra aprendizado algum.

Também pode haver **viés do próprio algoritmo preditivo** e **formas de agregar dados**, se isso for uma etapa o aprendizado.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

É crucial abordar esses tipos de viéses para garantir que os sistemas sejam justos e equitativos. Avaliações regulares e ajustes manuais são necessários para mitigar viéses indesejados.

5.2 Construindo soluções éticas com Inteligência Artificial

A avaliação ética em IA deve ser uma parte integrante do ciclo de vida do desenvolvimento. Isso inclui avaliar o impacto social, identificar e corrigir viéses, garantir transparência nos algoritmos e envolver partes interessadas em processos decisórios. Soluções éticas envolvem a diversificação de equipes de desenvolvimento para evitar monoculturas de pensamento, implementação de auditorias éticas regulares e o desenvolvimento de frameworks que priorizem a equidade e a justiça.

A tabela abaixo destaca algumas **estratégias-chave para a construção de soluções éticas em IA**:

Estratégias Éticas	Exemplos de Implementação
Diversificação de Dados	Inclusão de dados representativos de todas as comunidades.
Transparência Algorítmica	Utilização de algoritmos explicáveis e interpretação de resultados.
Envolvimento de Stakeholders	Consultas regulares com grupos afetados e incorporação de feedback.
Educação Ética	Treinamento de desenvolvedores e usuários sobre questões éticas em IA.

Tabela 1 – Como construir soluções éticas em IA?
Fonte: Autor (2023)

Em resumo, enquanto a IA oferece oportunidades emocionantes, a ética desempenha um papel central na garantia de que essas tecnologias sejam benéficas para a sociedade. Superar dilemas éticos, abordar viéses e incorporar princípios éticos desde o início são passos cruciais para moldar um futuro em que a IA promove o bem comum. O diálogo contínuo entre desenvolvedores, formuladores de políticas e sociedade é essencial para alcançar um equilíbrio ético duradouro.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

6 DESAFIOS ATUAIS E FUTUROS

A Inteligência Artificial (IA) enfrenta desafios substanciais, tanto presentes quanto futuros, enquanto continua a moldar nosso mundo.

Um desafio crítico é a questão da **ética e viés algorítmico**. Exemplos notáveis incluem sistemas de reconhecimento facial que demonstram precisões variáveis entre diferentes grupos étnicos, levantando preocupações sobre discriminação. Para superar esses desafios éticos, é imperativo desenvolver algoritmos mais transparentes e equitativos, garantindo representatividade nos conjuntos de dados.

Outro desafio atual é a **interpretabilidade dos modelos de IA**. A complexidade de algoritmos de aprendizado profundo muitas vezes os torna "caixas-pretas", dificultando a compreensão de como tomam decisões. Abordar esse desafio é crucial, especialmente em setores críticos como a saúde, onde a confiança nas decisões de IA é fundamental.

Quanto ao futuro, uma tendência importante é a **IA contextualizada**, que busca compreender melhor o ambiente e as nuances da linguagem. O **processamento de linguagem natural avançado**, exemplificado pelo modelo GPT-4, evidencia essa evolução. A IA também se movimenta em direção à integração mais estreita com a Internet das Coisas (IoT), possibilitando **sistemas mais inteligentes e autônomos**.

Para o sucesso dessas tendências, é imprescindível trabalharmos com MUITOS DADOS e técnicas robustas, como Deep Learning. Portanto, nosso próximo capítulo iniciará nossa jornada no mundo das redes neurais artificiais!

7 CIÊNCIA DE DADOS EM PYTHON: BIBLIOTECA SKLEARN

Agora que passamos pelos principais conceitos de aprendizado de máquina, está na hora de conhecermos as principais bibliotecas em Python que implementam os algoritmos preditivos. Começaremos com o **scikit-learn**, desenvolvendo na prática os algoritmos citados neste capítulo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

DICA: veja o material HOW TO para acessar o ambiente do Colab.

DICA: uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: aula 2) e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

7.1 Implementando algoritmos supervisionados: regressão

Vimos a intuição da regressão linear e da regressão múltipla. Agora, vamos criar nossos primeiros modelos preditivos de regressão.

DICA: para essa parte do código, utilize o arquivo de apoio “galton.csv”.

A base a ser utilizada é um conjunto de dados muito famoso! A base "galton.csv" tem origem nos estudos pioneiros do cientista Francis Galton no final do século XIX. Essa base, derivada de observações sobre estatura parental e filial, foi amplamente utilizada para explorar padrões de hereditariedade e fundamentou os primeiros desenvolvimentos em modelos estatísticos, incluindo a regressão linear, destacando-se como uma referência histórica na estatística e ciência de dados.

Vamos começar com os *imports*:

```
# Imports necessários
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
```

Código-fonte 1 – Imports para regressão linear

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Fonte: Elaborado pelo autor (2023)

Ao executar o código acima, você não deve observar saída alguma. Então, com os códigos importados, podemos fazer a leitura do arquivo CSV:

```
# Ler o dataset do arquivo CSV
df = pd.read_csv("galton.csv")
df.head(5)
```

Código-fonte 2 – Leitura do dataset
Fonte: Elaborado pelo autor (2023)

O código acima carrega o arquivo CSV na memória, salvando o conteúdo em um DataFrame Pandas chamado “df”, cujo conteúdo das 5 primeiras linhas é:

	family	father	mother	midparentHeight	children	childNum	gender	childHeight
0	1	78.5	67.0	75.43	4	1	male	73.2
1	1	78.5	67.0	75.43	4	2	female	69.2
2	1	78.5	67.0	75.43	4	3	female	69.0
3	1	78.5	67.0	75.43	4	4	female	69.0
4	2	75.5	66.5	73.66	4	1	male	73.5

Figura 11 – Resultado do código-fonte 2
Fonte: Elaborado pelo autor (2023)

A Figura 11 mostra as variáveis presentes da base. Temos, na ordem: o ID da família, a altura do pai, a altura da mãe, a “altura média parental”, o número de filhos, o ID de cada filho, o gênero desse filho e a altura do filho na idade adulta. Essa “altura média parental” na verdade é uma média ponderada das alturas do pai e da mãe.

Agora vejamos algumas informações dos dados:

```
# Infos do dataset
df.info()
```

Código-fonte 3 – Informações consolidadas do dataset
Fonte: Elaborado pelo autor (2023)

O código acima fornece uma visão geral do dataset, em termos de nomes e tipos das variáveis, quantidade de linhas e colunas, número de dados faltantes, entre

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

outras informações relevantes. Como mostra a Figura 12, estamos diante de um datast bem-comportado, sem dados faltantes.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 934 entries, 0 to 933
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   family           934 non-null    object
 1   father           934 non-null    float64
 2   mother           934 non-null    float64
 3   midparentHeight  934 non-null    float64
 4   children         934 non-null    int64
 5   childNum         934 non-null    int64
 6   gender           934 non-null    object
 7   childHeight      934 non-null    float64
dtypes: float64(4), int64(2), object(2)
memory usage: 58.5+ KB
```

Figura 12 – Resultado do código-fonte 3
Fonte: Elaborado pelo autor (2023)

Ainda assim, é bom fazermos alguns tratamentos nesses dados, por exemplo, eliminando dados duplicados:

```
# Tratamento de dados - REMOVENDO LINHAS DUPLICADAS
df.drop_duplicates(inplace=True)
df.shape
```

Código-fonte 4 – Eliminando dados duplicados
Fonte: Elaborado pelo autor (2023)

Esse procedimento é particularmente importante na criação de algoritmos preditivos para garantirmos que os subconjuntos de treino e teste sejam disjuntos. Como vemos a saída do “shape” do “df” após a limpeza, ele se mantém do mesmo tamanho (Figura 13). Ou seja, não há dados duplicados.

```
(934, 8)
```

Figura 13 – Resultado do código-fonte 4
Fonte: Elaborado pelo autor (2023)

Nosso problema preditivo é: *dadas as alturas dos pais, podemos prever a altura dos filhos na idade adulta?* Para responder a essa pergunta, é natural usarmos algumas variáveis dadas, mas outras não fazem sentido, justamente aquelas de “ID”. É como se, para responder nossa modelagem, olhássemos para o CPF ou RG da

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

pessoa! IDs não fazem sentido como variáveis preditoras em modelos preditivos. Portanto, vamos eliminar as colunas inúteis:

```
# Tratamento de dados - REMOVENDO COLUNAS INÚTEIS PARA O PROBLEMA
df.drop(columns=["family", "children", "childNum"], inplace=True)
```

Código-fonte 5 – Eliminando colunas inúteis
Fonte: Elaborado pelo autor (2023)

O código acima não tem retorno. Por quê? Porque usamos o parâmetro “inplace=True”, que altera a própria variável “df”. Podemos fazer outra alteração, mais significativa para compreensão da base, que é a conversão da unidade de medida de “polegadas” para “centímetros”. O código baixo define uma função com essa transformação e a aplica em todas as colunas que medem altura de pessoas:

```
# Tratamento de dados - PASSANDO DE INCHES PARA CM (1 in = 2.54 cm)
def inch2cm(inch):
  return inch*2.54

for coluna in ["father", "mother", "midparentHeight", "childHeight"]:
  df[coluna] = df[coluna].apply( inch2cm )

df.sample(10)
```

Código-fonte 6 – Alterando o sistema métrico
Fonte: Elaborado pelo autor (2023)

Nosso último bloco de código termina exibindo 10 linhas aleatórias do DataFrame resultante. Sua saída deve ser similar à da Figura 14, mas provavelmente não será igual:

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	father	mother	midparentHeight	gender	childHeight
78	182.880	172.72	184.7088	female	172.720
923	162.560	162.56	169.0624	female	152.400
902	166.370	160.02	169.5958	male	180.340
897	165.100	160.02	168.9608	female	160.020
770	170.180	161.29	172.1866	male	166.370
858	167.640	149.86	164.7444	female	144.780
413	175.260	173.99	181.5846	male	180.340
686	174.498	157.48	172.2882	male	171.958
890	165.100	162.56	170.3324	male	170.180
238	177.800	175.26	183.5404	male	175.260

Figura 14 – Resultado do código-fonte 6
Fonte: Elaborado pelo autor (2023)

A representação dos dados presente na Figura 14 mostra que eles já estão quase no ponto de alimentar modelos preditivos. A última transformação a ser feita é passar a coluna “gender” de string para valores numéricos. Podemos fazer isso de diversas formas, uma delas é por meio do código a seguir:

```
# Tratamento de dados - PASSANDO "GENDER" DE STRING PARA NÚMERO
df["gender"] = df["gender"].apply( lambda x: 0 if x=="male" else 1 )
df.sample(10)
```

Código-fonte 7 – Convertendo a coluna “gender”
Fonte: Elaborado pelo autor (2023)

Novamente, sua saída deve ser similar à da Figura 15, mas potencialmente não idêntica.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	father	mother	midparentHeight	gender	childHeight
718	170.180	168.148	175.88992	0	184.658
410	177.800	147.320	168.45280	1	165.100
256	177.800	168.910	180.11140	0	168.910
228	180.340	157.480	175.20920	0	175.260
522	175.260	157.480	172.66920	0	180.340
359	179.070	160.020	175.94580	1	166.370
870	165.100	167.640	173.07560	0	165.100
178	182.118	163.830	179.52720	1	170.180
841	167.640	152.400	166.11600	0	165.100
279	177.800	165.100	178.05400	1	160.020

Figura 15 – Resultado do código-fonte 7
Fonte: Elaborado pelo autor (2023)

Como mencionado anteriormente, essa base é bem-comportada, não possuindo valores duplicados ou faltantes. Ainda assim, pode ser uma boa hipótese criar uma nova variável com o valor médio exato entre as alturas do pai e da mãe. Isso é feito facilmente via Pandas:

```
# Tratamento de dados - CRIAR UMA COLUNA NOVA COM A MÉDIA DA ALTURA
DOS PAIS
df["media"] = ( df["father"] + df["mother"] ) / 2
```

Código-fonte 8 – Criando a coluna “média”
Fonte: Elaborado pelo autor (2023)

Ótimo! Agora estamos num bom ponto para criar nosso primeiro modelo de predição! Iniciaremos com um modelo de regressão linear simples, correlacionando apenas a altura “média” dos pais (nossa variável X) com a altura dos filhos (nosso *label*, variável Y).

ATENÇÃO: como estamos trabalhando com duas variáveis e que estão expressas na mesma escala, nesse caso, dispensa-se a etapa de normalização/padronização de features!

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Separação de DADOS e LABELS
X = np.array( df["media"] ).reshape(-1, 1) # Reshape pois o dataframe
tem apenas 1 coluna
y = df["childHeight"]

# Separação de TREINO e TESTE
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size=0.3,
                                                    random_state=42)
```

Código-fonte 9 – Separação de dados em “treino” e “teste”
Fonte: Elaborado pelo autor (2023)

Agora vamos finalmente usar a biblioteca scikit-learn para instanciar o algoritmo de regressão linear e treiná-lo, ou seja, encontrar a reta que modela a relação linear entre altura média dos pais e altura dos filhos na idade adulta. Nessa biblioteca, o processo de (a) instanciar o algoritmo preditivo, (b) treiná-lo e (c) gerar predições no teste precisa de apenas 3 linhas!

```
# Treinar o modelo usando os dados de TREINO
reta = LinearRegression()
reta.fit(X_train, y_train)

y_pred = reta.predict(X_test)
```

Código-fonte 10 – Criando o modelo de regressão linear simples
Fonte: Elaborado pelo autor (2023)

Como nosso primeiro modelo performa? Para responder essa questão, temos que empregar as métricas de avaliação do modelo! Conforme visto anteriormente, vamos avaliar o modelo em termos de MSE, MAE e R^2. O scikit-learn também possui implementações (já importadas no início do código) que facilitam os cálculos:

```
# Avaliar o modelo nos dados de TESTE
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MSE = {mse}")
print(f"MAE = {mae}")
print(f"R² = {r2}")
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Código-fonte 11 – Avaliando o modelo de regressão linear simples
Fonte: Elaborado pelo autor (2023)

As métricas estão na Figura 16. Podemos ver que esse modelo não está muito bom, afinal, o MAE é de ~7cm, ou seja, para cada predição ele erra na média 7 cm pra cima ou pra baixo! O erro quadrático médio (MSE) está em 76,5 cm! E para confirmar ainda mais que esse não é um bom modelo, note que o R^2 está apenas com 0,13 (e deveria ser próximo de 1).

```
MSE = 76.51508748754067
MAE = 7.40918070892715
R² = 0.12838023412454402
```

Figura 16 – Resultado do código-fonte 11
Fonte: Elaborado pelo autor (2023)

O que será que explica os resultados baixos? Bom, podemos tentar encontrar uma explicação visual para o problema. Vamos plotar os dados de teste e traçar, sobre eles, o modelo que simula a relação linear entre as variáveis:

```
# Gráfico da regressão simples
import matplotlib.pyplot as plt

plt.scatter(X_test, y_test)
plt.plot(X_test, y_pred, color="red")
plt.title("Regressão da altura média dos pais e dos filhos")
plt.xlabel("Altura média dos pais (cm)")
plt.ylabel("Altura dos filhos (cm)")
plt.show()
```

Código-fonte 12 – Plotando os dados de teste e o modelo resultante
Fonte: Elaborado pelo autor (2023)

A Figura 17 mostra a saída da última célula de código. Por esse gráfico podemos ver que é possível notar uma relação linear crescente entre as alturas médias dos pais e a altura dos filhos, no entanto, muitos pontos possuem erros altos quando comparado o valor predito pelo modelo com o valor real (distância vertical entre o ponto e a reta). Note também que há diversos pontos “outliers”, ou seja, que fogem da distribuição, com filhos muito altos ou muito baixos. Quando tentamos

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

simular todos esses exemplos com um modelo, é normal que ele não se ajuste bem aos dados.

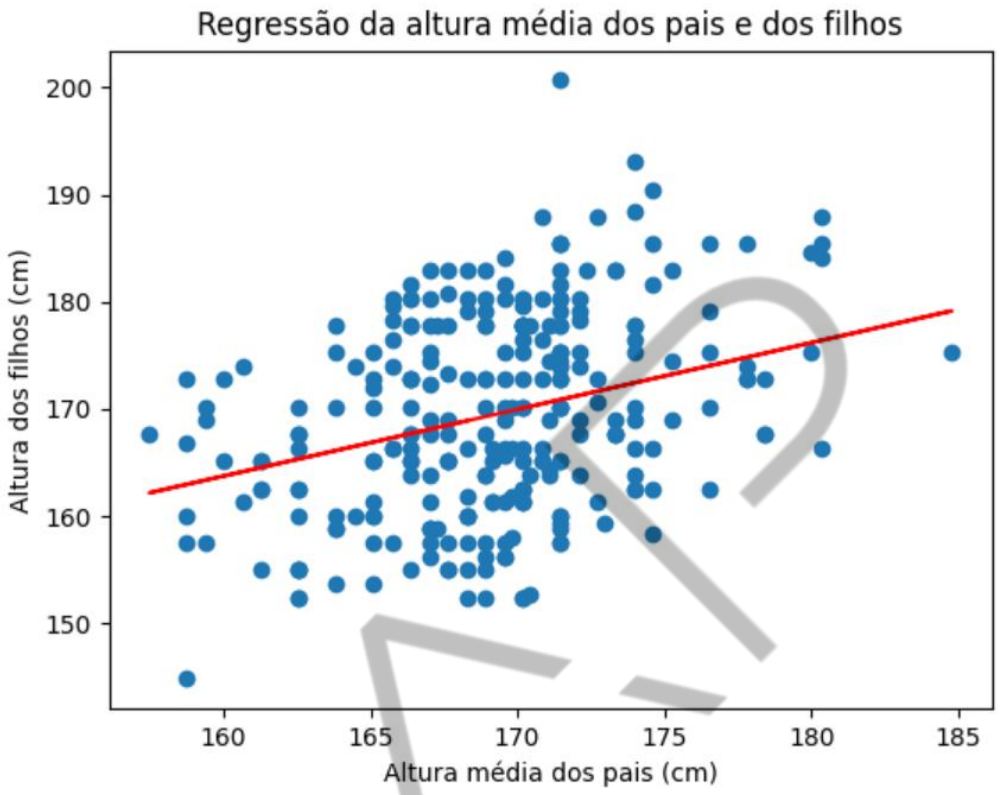


Figura 17 – Resultado do código-fonte 12
Fonte: Elaborado pelo autor (2023)

Será que podemos construir um modelo melhor? Sabemos que de forma geral, homens tendem a ser mais altos que mulheres, portanto, incorporar o sexo biológico (“gender”) no modelo pode ser uma boa pista para a regressão ser mais bem construída. Como usaremos duas variáveis preditoras (X1, X2), estaremos criando nosso primeiro modelo de regressão linear múltipla.

Vamos começar novamente separando os dados de treinamento e teste. Como usamos a mesma semente aleatória, garantimos que os resultados sejam comparáveis com os do primeiro modelo.

```
# Separação de DADOS e LABELS
X = df.drop(columns=["childHeight", "media"])
y = df["childHeight"]

# Separação de TREINO e TESTE
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
                                                                    test_size=0.3,
                                                                    random_state=42)
```

Código-fonte 13 – Separação de dados em “treino” e “teste” para a regressão múltipla
Fonte: Elaborado pelo autor (2023)

E agora sim, cada variável está expressa em uma escala! Temos a altura média dos pais em centenas e o gênero em unidades. Portanto, é imprescindível normalizar/padronizar os dados:

```
# Padronizar os dados
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Código-fonte 14 – Normalização dos dados
Fonte: Elaborado pelo autor (2023)

Agora sim vamos gerar um novo modelo preditivo:

```
# Treinar o modelo
reta = LinearRegression()
reta.fit(X_train_scaled, y_train)

y_pred = reta.predict(X_test_scaled)
```

Código-fonte 15 – Criando o modelo de regressão linear múltipla
Fonte: Elaborado pelo autor (2023)

E em seguida avaliá-lo sob as métricas vistas anteriormente:

```
# Avaliar o modelo
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MSE = {mse}")
print(f"MAE = {mae}")
print(f"R² = {r2}")
```

Código-fonte 16 – Avaliando o modelo de regressão linear múltipla
Fonte: Elaborado pelo autor (2023)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

E o novo resultado? Muito melhor, como mostra a Figura 18! Note que o R^2 está maior e mais próximo de 1, e o erro absoluto (MAE) reduziu para 4 cm, assim, cada predição levará a um erro de 4 cm para cima ou para baixo da altura real do filho.

```
MSE = 30.684177685091104
MAE = 4.349358693772597
R² = 0.6504619330884904
```

Figura 18 – Resultado do código-fonte 16
Fonte: Elaborado pelo autor (2023)

7.2 Implementando algoritmos supervisionados: classificação

Vimos a intuição do método do KNN como exemplo de algoritmo para classificação. Agora vamos criar nossos primeiros modelos preditivos de classificação!

DICA: para essa parte do código, utilizaremos um dataset já salvo na memória do scikit-learn, por isso não há um arquivo CSV associado!

Vamos utilizar a base de vinhos já persistida no scikit-learn. Sua origem remonta aos estudos sobre análise química de vinhos cultivados em uma mesma região da Itália, realizados por Forina et al. em 1987. Essa base de dados é comumente usada como referência para modelos de classificação, explorando características químicas para distinguir entre diferentes variedades de vinho, proporcionando um cenário desafiador e prático para avaliação de algoritmos de aprendizado de máquina na classificação de múltiplas classes.

Vamos começar com os *imports*:

```
# Imports necessários
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
from sklearn import datasets

import matplotlib.pyplot as plt

import pandas as pd
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
import numpy as np
```

Código-fonte 17 – Imports para classificação com KNN
Fonte: Elaborado pelo autor (2023)

Em seguida, vamos carregar o dataset e armazená-lo em um DataFrame Pandas chamado “df”:

```
# Obtendo o dataset
tabela = datasets.load_wine()
df = pd.DataFrame(tabela.data, columns=tabela.feature_names)
df["label"] = tabela.target
df
```

Código-fonte 18 – Estruturando os dados como um DataFrame Pandas
Fonte: Elaborado pelo autor (2023)

Ao executar o código acima, você verá um DataFrame “grande”, com 178 linhas (exemplos) e 14 colunas (variáveis), parcialmente ilustrado na Figura 19.

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69
...	...	...	...	...	...	...	...
173	13.71	5.65	2.45	20.5	95.0	1.68	0.61
174	13.40	3.91	2.48	23.0	102.0	1.80	0.75
175	13.27	4.28	2.26	20.0	120.0	1.59	0.69
176	13.17	2.59	2.37	20.0	120.0	1.65	0.68
177	14.13	4.10	2.74	24.5	96.0	2.05	0.76

178 rows × 14 columns

Figura 19 – Resultado do código-fonte 18
Fonte: Elaborado pelo autor (2023)

Seguindo o pipeline de Ciência de Dados, deveríamos explorar e limpar essa base, assim como fizemos com o modelo de regressão. Mas, já adianto que esta

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

também é uma base bem-comportada, sem valores duplicados ou faltantes. A exploração da base fica a cargo do leitor! Vamos direto para a modelagem preditiva!

Nosso modelo preditivo será simples: *dadas todas essas características de vinhos, conseguimos predizer seu tipo (sua classe)*? Para isso, todas as variáveis serão features.

NOTA: nem sempre usar todas as variáveis é o melhor caminho para construir modelos preditivos. Um bom entendimento do negócio e uma análise exploratória dos dados podem lhe ajudar e decidir quais variáveis manter como preditoras e quais excluir.

```
# Dividir o dataset em TREINO e TESTE
X = df.drop(columns=["label"])
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)
```

Código-fonte 19 – Separação de dados em “treino” e “teste” para a classificação
Fonte: Elaborado pelo autor (2023)

Como estamos trabalhando com diversas variáveis, cada uma em uma escala, é ESSENCIAL padronizar/normalizar os dados. Aqui, aplicaremos as duas transformações de modo independentes para comparar os resultados!

```
# Padronizar e Normalizar os dados
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

mm = MinMaxScaler()
X_train_norm = mm.fit_transform(X_train)
X_test_norm = mm.transform(X_test)
```

Código-fonte 20 – Padronização e Normalização de dados
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Como vimos na regressão, o scikit-learn possui uma estrutura muito direta para treinamento e uso de modelos preditivos. No caso do KNN não é diferente. Mas nós queremos avaliar os resultados de acordo com alterações no valor de K. Portanto, para economizar código e tornar nosso script mais organizado, criaremos uma função chamada “meu_knn()” que recebe dados e labels do treino e do teste, bem como um valor de K, para construir o modelo e avaliá-lo.

```
# Criar uma função que aplica e avalia o KNN
def meu_knn(xtr, xts, ytr, yts, k=0):

  # Definir o modelo
  if k == 0:
    k = int( (len(ytr) + len(yts)) ** 0.5 )

  # Construtor do método pelo Sklearn
  knn = KNeighborsClassifier(n_neighbors=k)

  # Treina o modelo nos dados de treino e seu label
  knn.fit(xtr, ytr)

  # Avalia o modelo, por meio de predições no teste
  y_pred = knn.predict(xts)
  acc = metrics.accuracy_score(yts, y_pred)
  print(f"k={k}\tAcurácia: {acc:.4f}")
```

Código-fonte 21 – Função que aplica e avalia o KNN em função do valor de K
Fonte: Elaborado pelo autor (2023)

O código acima possui um parâmetro opcional: o valor de K! Se ele não for informado no momento da chamada da função “meu_knn()”, nós utilizaremos o valor default “0”. É impossível usar o KNN com K=0, portanto, usaremos esse valor como uma indicação que devemos aplicar uma heurística para encontrar um valor de K “ideal”. Na teoria do KNN, existe uma recomendação que um “bom” valor aproximado para K é a raiz quadrada do tamanho do dataset. Calculamos isso somando o tamanho das listas de labels do treino com a lista de labels do teste e elevando a 0,5, que é equivalente a extrair a raiz quadrada de um número. Como o resultado pode ser um valor quebrado (e não faz sentido consultar 3,878 vizinhos), nós arredondamos o valor para um inteiro.

Agora, podemos chamar nossa função customizada em diversas configurações. Comecemos pelo emprego dos dados originais:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Avaliando o KNN com diferentes valores de K, a partir dos dados
ORIGINAIS
meu_knn(X_train, X_test, y_train, y_test, 5) # k=5
meu_knn(X_train, X_test, y_train, y_test, 7) # k=7
meu_knn(X_train, X_test, y_train, y_test, 9) # k=9
meu_knn(X_train, X_test, y_train, y_test)    # k=determinado
automaticamente
```

Código-fonte 22 – Aplicando “meu_knn()” sobre os dados originais
Fonte: Elaborado pelo autor (2023)

Vamos aos resultados? A Figura 20 mostra o que esperávamos: a escolha do valor de K altera a métrica (acurácia). O valor default de K, para esse conjunto de dados, é 13, afinal, a raiz quadrada de 178 é ~13,342 que é “13” no formato *int*.

```
k=5     Acurácia: 0.7407
k=7     Acurácia: 0.7593
k=9     Acurácia: 0.7222
k=13    Acurácia: 0.7593
```

Figura 20 – Resultado do código-fonte 22
Fonte: Elaborado pelo autor (2023)

Mas o que acontece se, ao invés dos dados originais, usarmos os dados PADRONIZADOS? Note aqui a beleza e facilidade de aplicarmos a função customizada nesse processo! Economizamos várias linhas repetitivas de “fit()” e “predict()”.

```
# Avaliando o KNN com diferentes valores de K, a partir dos dados
PADRONIZADOS
meu_knn(X_train_scaled, X_test_scaled, y_train, y_test, 5) # k=5
meu_knn(X_train_scaled, X_test_scaled, y_train, y_test, 7) # k=7
meu_knn(X_train_scaled, X_test_scaled, y_train, y_test, 9) # k=9
meu_knn(X_train_scaled, X_test_scaled, y_train, y_test)    #
k=determinado automaticamente
```

Código-fonte 23 – Aplicando “meu_knn()” sobre os dados padronizados
Fonte: Elaborado pelo autor (2023)

E os resultados? Eles melhoraram, como mostra a Figura 21!

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
k=5      Acurácia: 0.9630
k=7      Acurácia: 0.9815
k=9      Acurácia: 0.9815
k=13     Acurácia: 0.9630
```

Figura 21 – Resultado do código-fonte 23
Fonte: Elaborado pelo autor (2023)

E em termos de dados NORMALIZADOS?

```
# Avaliando o KNN com diferentes valores de K, a partir dos dados
NORMALIZADOS
meu_knn(X_train_norm, X_test_norm, y_train, y_test, 5) # k=5
meu_knn(X_train_norm, X_test_norm, y_train, y_test, 7) # k=7
meu_knn(X_train_norm, X_test_norm, y_train, y_test, 9) # k=9
meu_knn(X_train_norm, X_test_norm, y_train, y_test)    # k=determinado
automaticamente
```

Código-fonte 24 – Aplicando “meu_knn()” sobre os dados normalizados
Fonte: Elaborado pelo autor (2023)

A Figura 22 mostra que eles melhoraram ainda mais!

```
k=5      Acurácia: 0.9444
k=7      Acurácia: 0.9630
k=9      Acurácia: 0.9815
k=13     Acurácia: 0.9815
```

Figura 22 – Resultado do código-fonte 24
Fonte: Elaborado pelo autor (2023)

Para não haver dúvidas, vamos centralizar na mesma célula os resultados do mesmo experimento (mesmo valor de K) nas três configurações de dados:

```
# Comparando os 3 métodos com o valor de "K" padrão.
meu_knn(X_train, X_test, y_train, y_test)               # Dados
originais
meu_knn(X_train_scaled, X_test_scaled, y_train, y_test) # Dados
padronizados
meu_knn(X_train_norm, X_test_norm, y_train, y_test)     # Dados
normalizados
```

Código-fonte 25 – Comparação de execuções
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A Figura 23 consolidada os resultados reforçando que a normalização, aqui, foi melhor que a padronização:

```
k=13    Acurácia: 0.7593
k=13    Acurácia: 0.9630
k=13    Acurácia: 0.9815
```

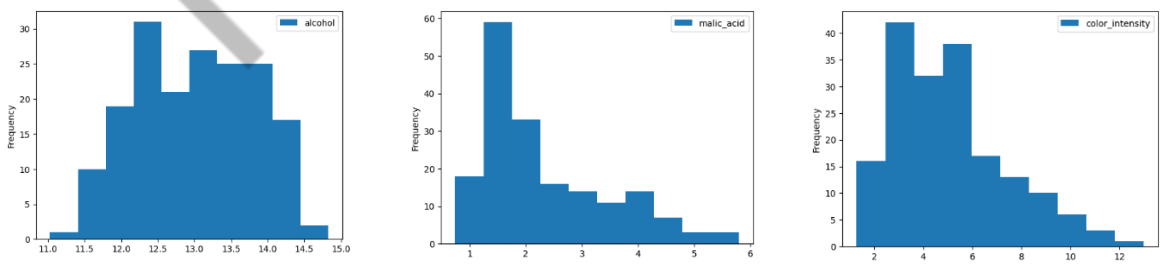
Figura 23 – Resultado do código-fonte 25
Fonte: Elaborado pelo autor (2023)

Você está se perguntando por que isso acontece! Isso está relacionado com a distribuição das features. Nós aprendemos que a padronização (StandardScaler()) está mais voltada a escalar variáveis que seguem a distribuição normal. Quando os dados não seguem a destruição normal, é mais aconselhável aplicar a normalização (MinMaxScaler()).

Para avaliar a distribuição das features originais, execute o seguinte código e observe os gráficos gerados. Você verá que apesar de algumas features serem “normais”, a maioria delas não é. Logo, a normalização (MinMaxScaler()) leva a um resultado mais exato.

```
# Verificando a distribuição dos dados.
for col in df.columns:
  df[col].plot.hist()
  plt.legend()
  plt.show()
```

Código-fonte 26 – Construindo histogramas de cada feature do dataset
Fonte: Elaborado pelo autor (2023)



PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

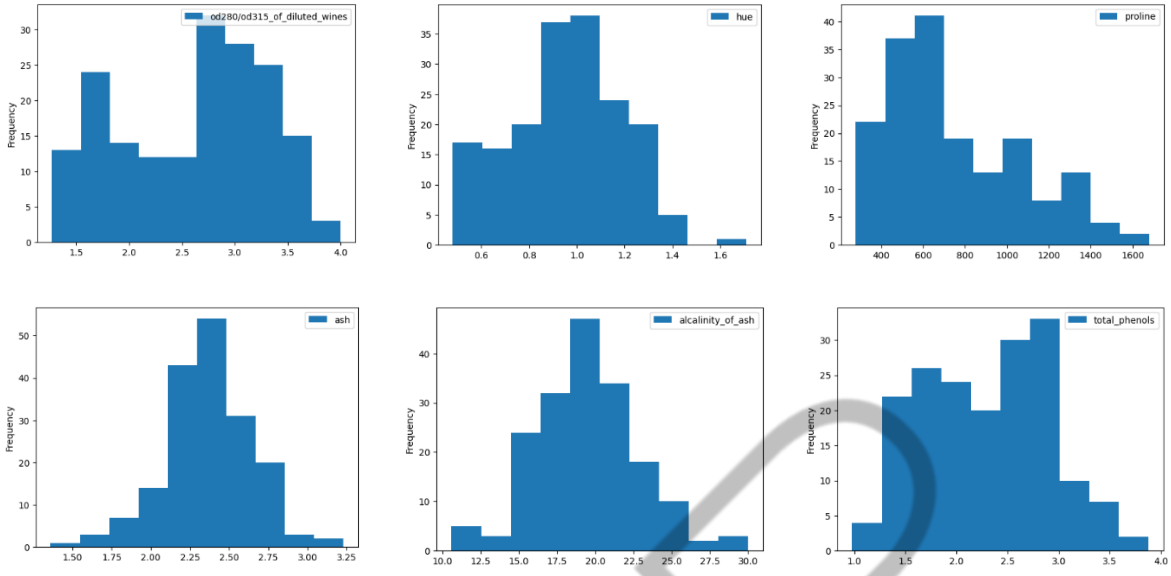


Figura 24 – Resultado parcial do código-fonte 26 (alguns histogramas foram omitidos)
Fonte: Elaborado pelo autor (2023)

Esse experimento reforça a importância de:

- Padronizar/normalizar features;
- Conhecer a distribuição das features antes de criar modelos preditivos.

7.3 Implementando algoritmos não-supervisionados: clusterização

Vimos a intuição do método do K-Means como exemplo de algoritmo não-supervisionado. Agora vamos criar nosso primeiro modelo de clusterização!

DICA: para essa parte do código, utilize o arquivo de apoio “mall.csv”.

A base "mall.csv" é originária de estudos sobre comportamento de clientes em shopping centers. Coletada a partir de registros de compras e interações, essa base de dados é frequentemente utilizada como referência para modelos de clusterização. Explorando padrões de preferências e comportamentos de compras, oferece insights valiosos para estratégias de segmentação de clientes em ambientes de varejo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Vamos começar com os *imports*:

```
# Imports necessários
import pandas as pd

import matplotlib.pyplot as plt
import matplotlib
from matplotlib import colors
import seaborn as sns

from sklearn.cluster import KMeans
```

Código-fonte 27 – Imports da clusterização com K-Means
Fonte: Elaborado pelo autor (2023)

Agora vamos ler os dados:

```
dados = pd.read_csv("mall.csv")
dados.head()
```

Código-fonte 28 – Leitura da base “mal.csv”
Fonte: Elaborado pelo autor (2023)

Percebemos pela Figura 25 que o dataset é formado por 5 colunas: ID do cliente, gênero, idade, renda anual e score de gastos, respectivamente.

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Figura 25 – Resultado do código-fonte 28
Fonte: Elaborado pelo autor (2023)

Para conhecer um pouco mais dos dados, o “info()” é sempre um bom princípio:

```
dados.info()
```

Código-fonte 29 – Explorando brevemente a base “mal.csv”
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Aqui também estamos diante de uma base pequena (200 exemplos), sem dados faltantes e sem grandes necessidades de alterações dos tipos de dados:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

Figura 26– Resultado do código-fonte 29
Fonte: Elaborado pelo autor (2023)

Vamos explorar visualmente essa base por meio do pairplot do Seaborn:

```
sns.pairplot(dados)
plt.show()
```

Código-fonte 30 – Explorando brevemente a base “mal.csv”
Fonte: Elaborado pelo autor (2023)

A Figura 26 mostra o resultado do comando acima. A diagonal principal mostra o histograma de cada variável e as demais posições os gráficos resultantes do plot pareado, duas a duas variáveis.

Como é de se esperar, alguns plots não mostram insights relevantes (como qualquer plot com ID do cliente). Outros não mostram tendências claras: por exemplo, ao cruzar ganho anual com idade, não há uma relação óbvia, afinal, podemos ter pessoas que ganham bem ou mal em qualquer faixa etária.

No entanto, plots que cruzam rendimento anual com score de gasto mostram uma tendência interessante: note que parece haver cinco grupos de usuários. Vamos criar uma clusterização para explorar esses grupos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

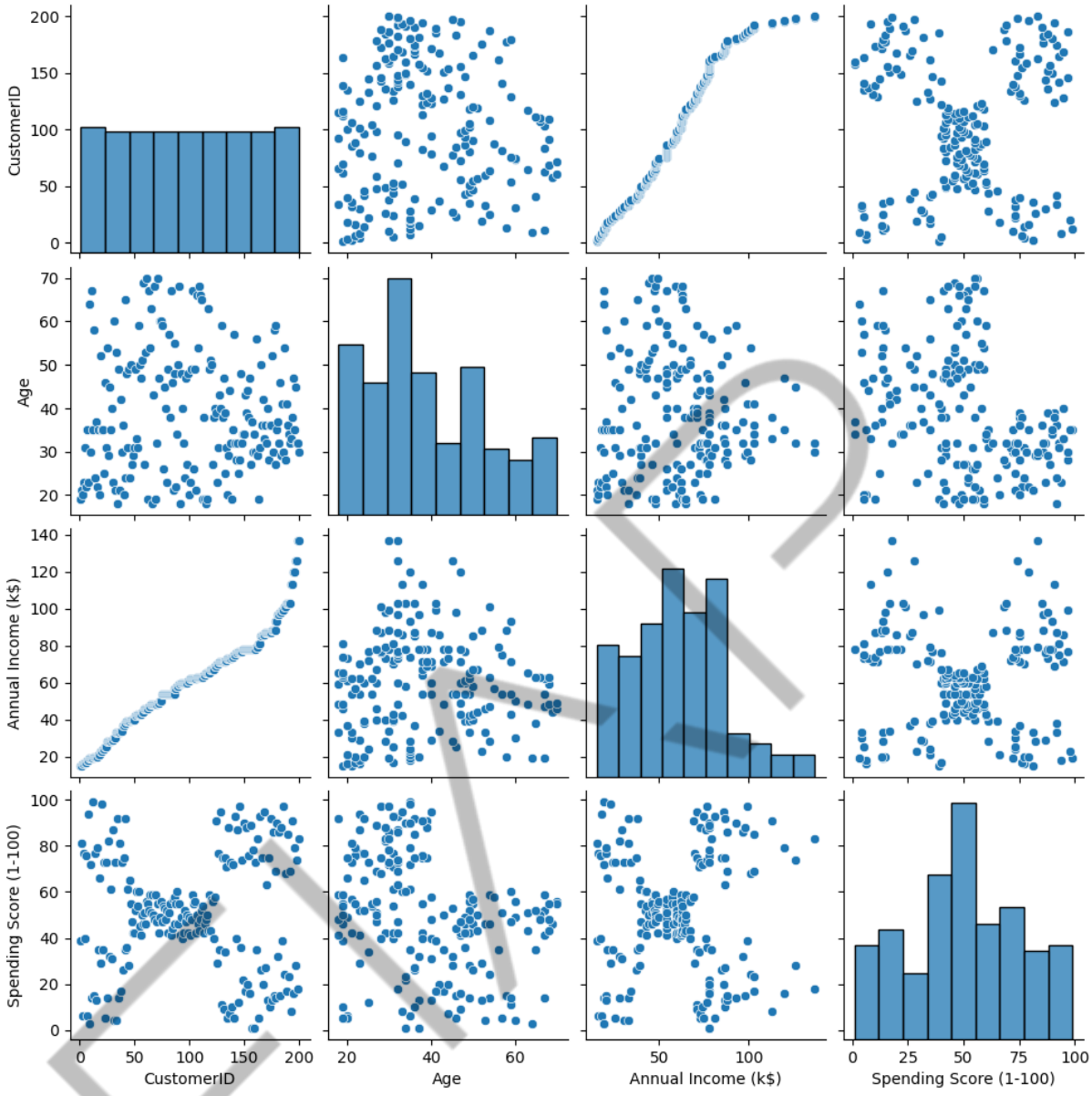


Figura 27 – Resultado do código-fonte 30
Fonte: Elaborado pelo autor (2023)

Nossa próxima célula de código já instancia o algoritmo do K-Means, solicitando o agrupamento em três grupos. Depois ajustamos os dados a esses grupos (fit()), salvamos os centroides dos grupos e a atribuição de grupo para cada exemplo.

```
# Instancia o algoritmo
kmeans = KMeans(n_clusters=3, random_state=0)

# Implementa o kmeans sobre os dados
kmeans.fit( dados[ ["Annual Income (k$)", "Spending Score (1-100)"] ]
)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Obtém os centroids do método
centroids = kmeans.cluster_centers_

# Salva os labels (grupos) dos clusteres para cada exemplo
kmeans_labels = kmeans.predict( dados[ ["Annual Income (k$)",
                                         "Spending Score (1-100)"] ] )
```

Código-fonte 31 – Aplicando o K-Means
Fonte: Elaborado pelo autor (2023)

Feito isso, podemos plotar os resultados por meio do código abaixo:

```
# Plotando a clusterização
plt.scatter(dados["Annual Income (k$)"],        # Eixo X
            dados["Spending Score (1-100)"],    # Eixo Y
            c=kmeans_labels,                    # Esquema de cores
(quantas usar)
            alpha=0.5,                          # Transparência dos
pontos
            cmap="rainbow")                     # Paleta de cores
plt.xlabel("Salário anual")
plt.ylabel("Pontuação de gastos")

# Plotando os centroids
plt.scatter(centroids[:, 0],                    # Eixo X dos centroids
            centroids[:, 1],                    # Eixo Y dos centroids
            c="black",                          # Todos os centroids
são pretos
            marker="X",                         # A marcação é um X
            s=200,                              # Tamanho da marcação
            alpha=0.5)

plt.show()
```

Código-fonte 32 – Gerando visualização dos clusteres formados
Fonte: Elaborado pelo autor (2023)

O código acima produz o gráfico representado na Figura 28. Como esperado, temos três grupos. Cada grupo comporta determinados exemplos da base (linhas de dados), e temos a visualização dos centroides.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

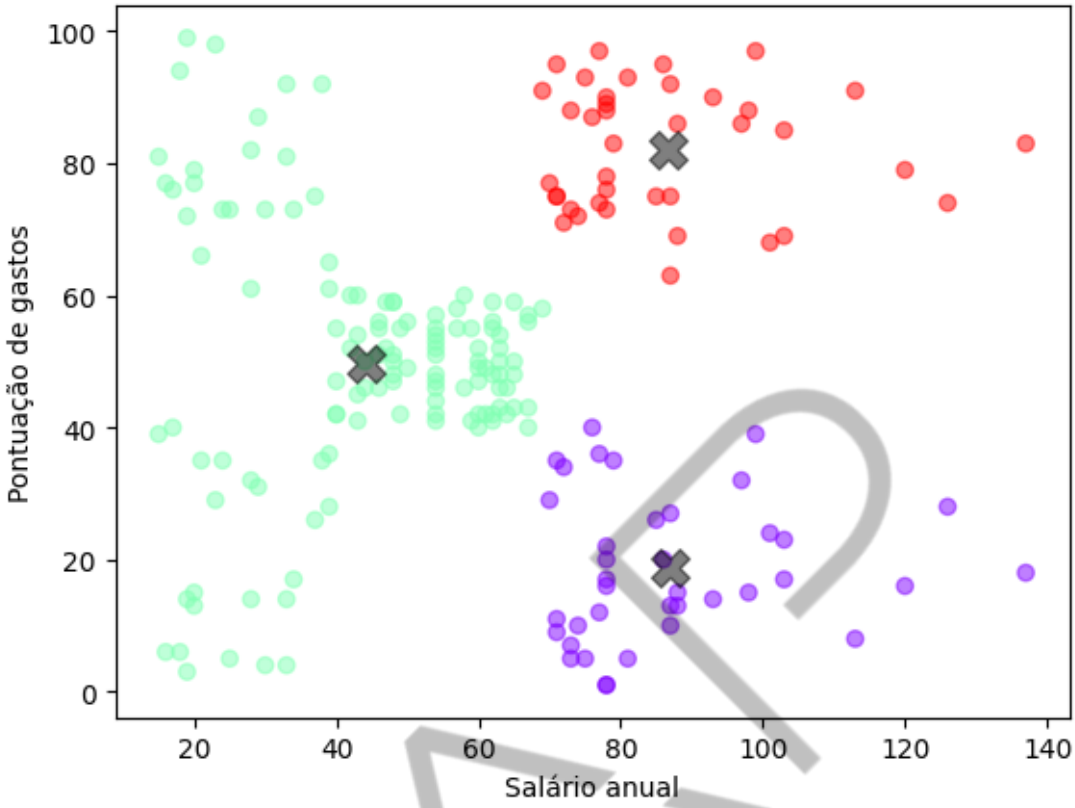


Figura 28 – Resultado do código-fonte 32
Fonte: Elaborado pelo autor (2023)

Nós vimos na introdução teórica do K-Means, que a escolha do K ideal é um trabalho subjetivo, mas que pode ser direcionado por meio de heurísticas, como o método do cotovelo. O código abaixo faz essa análise e gera o gráfico de análise:

```
# Escolha do K pelo método do cotovelo
k = list(range(1, 10))

sse = []
for i in k:
  kmeans = KMeans(n_clusters=i, random_state=0)
  kmeans.fit( dados[ ["Annual Income (k$)", "Spending Score (1-100)"]
] )
  sse.append( kmeans.inertia_ )

plt.plot(k, sse, "-o");
plt.title("Método do cotovelo")
plt.xlabel("Número de clusteres")
plt.ylabel("Inércia (SSE)")
plt.show();
```

Código-fonte 33 – Aplicação do Método do Cotovelo
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O código acima produz o gráfico representado na Figura 29. Como ilustrado no pairplot do Seaborn, os dados seriam mais bem divididos em 5 grupos. O método do cotovelo afirma que podemos ir além de 3 clusteres, pois a inércia melhoraria (diminuiria) se escolhêssemos 4 ou 5 clusteres. O ganho ao passar de 5 para 6 clusteres pode não ser justificável, pois a diminuição da inércia é bem menor do que nos passos anteriores.

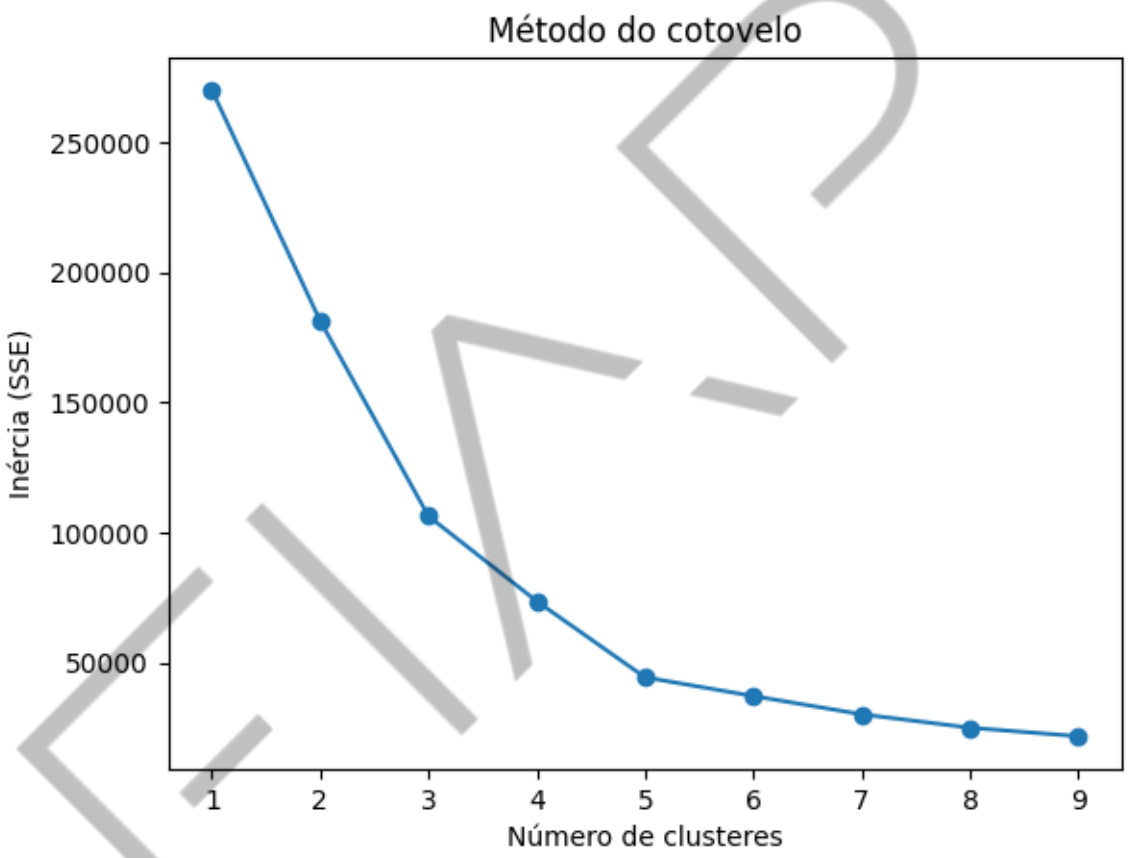


Figura 29– Resultado do código-fonte 33
Fonte: Elaborado pelo autor (2023)

Agora é com você! Volte no código 31 e altere o número de clusteres para 5 (*n_clusters=5*), execute novamente e rode também o código 32. Você deverá encontrar um gráfico similar a este:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

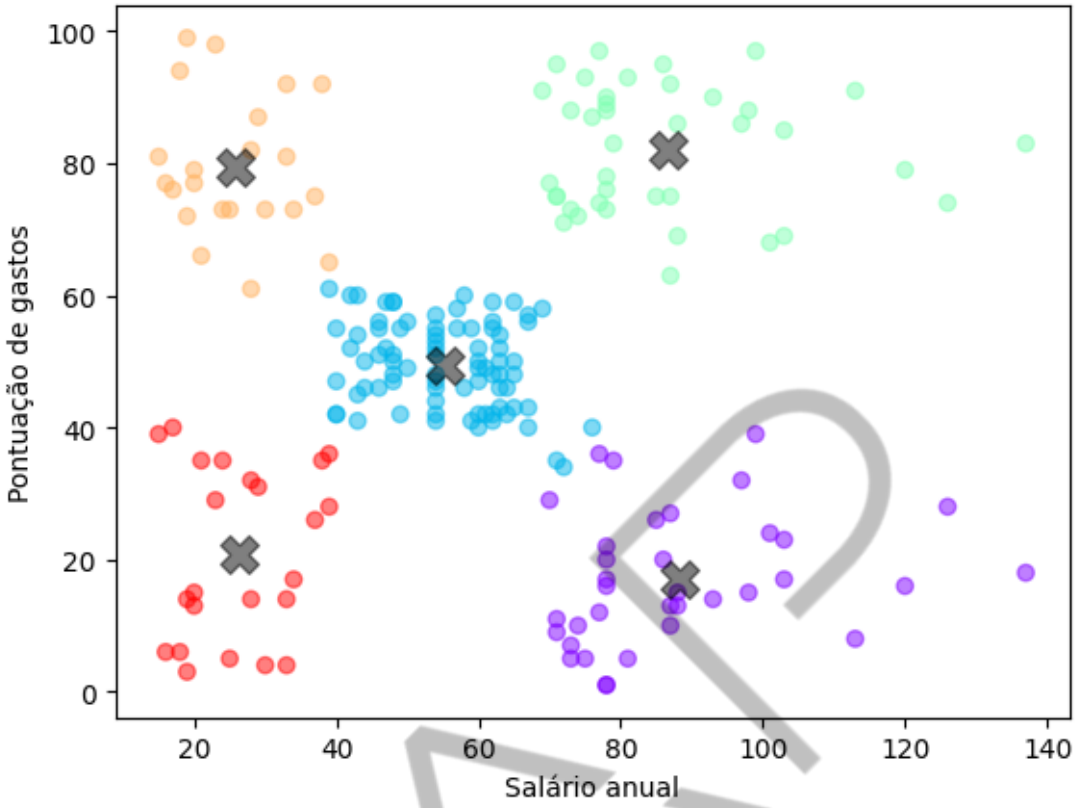


Figura 30– Resultado do código-fonte 32 adaptado para 5 clusteres
Fonte: Elaborado pelo autor (2023)

A Figura 30 mostra que há 5 perfis de clientes do shopping:

- Em amarelo: quem ganha pouco e gasta muito (se endivida);
- Em vermelho: quem ganha pouco e gasta pouco (prudente);
- Em verde: quem ganha muito e gasta muito (esbanja);
- Em roxo: quem ganha muito e gasta pouco (quem poupa);
- Em azul: quem gasta proporcional ao que ganha (perfil comum).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

8 HANDS ON: CONSTRUINDO NOVOS MODELOS DE REGRESSÃO, CLASSIFICAÇÃO E CLUSTERIZAÇÃO

8.1 Seu desafio

Agora é a sua vez de criar modelos de regressão, classificação e clusterização. Utilizando como base os códigos vistos no material, e as indicações de datasets, sua tarefa será:

1. **Para o modelo de regressão (base "galton.csv")**: vimos que adicionar a variável "gender" aprimorou o modelo. Será que outras relações matemáticas entre as alturas dos pais (além da média) seria melhor preditora para a altura dos filhos na idade adulta? Faça novos modelos explorando outras combinações de features (somando, subtraindo, extraindo o log etc.) e avalie se seu modelo ficou melhor ou pior que o modelo de regressão múltipla previamente construído;
2. **Para o modelo de classificação (base de vinhos)**: aqui nós temos diversas variáveis e utilizamos todas elas para montar nosso modelo inicial. Refaça o modelo preditivo utilizando apenas partes das variáveis que julgar mais relevantes. Avalie se o modelo ficou melhor ou pior que o modelo construído em conjunto;
3. **Para o modelo de clusterização (base "mal.csv")**: vamos dar um passo além da clusterização. Com os 5 clusteres já formados, utilize o ID do cluster para filtrar os consumidores e descreva o perfil de cada grupo. Para isso, faça seleções de dados via Pandas e empregue métodos descritivos como "value_counts()" e "describe()". A implementação é livre!

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

8.2 Proposta de solução

Abaixo, temos uma proposta de solução para os desafios apresentados. Essa é apenas uma forma de resolver, existem outras também! Qualquer dúvida, entre em contato com seu tutor.

REGRESSÃO:

A célula abaixo apenas repete os passos iniciais do código de regressão, lendo os dados, e fazendo os tratamentos necessários. Também já removemos colunas inúteis e passamos os dados de polegadas para centímetros.

```
# Imports necessários
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score

# Ler o dataset do arquivo CSV
df = pd.read_csv("galton.csv")
df.head(5)

# Tratamento de dados - REMOVENDO LINHAS DUPLICADAS
df.drop_duplicates(inplace=True)
df.shape

# Tratamento de dados - REMOVENDO COLUNAS INÚTEIS PARA O PROBLEMA
df.drop(columns=["family", "children", "childNum"], inplace=True)

# Tratamento de dados - PASSANDO DE INCHES PARA CM (1 in = 2.54 cm)
def inch2cm(inch):
  return inch*2.54

for coluna in ["father", "mother", "midparentHeight", "childHeight"]:
  df[coluna] = df[coluna].apply( inch2cm )

# Tratamento de dados - PASSANDO "GENDER" DE STRING PARA NÚMERO
df["gender"] = df["gender"].apply( lambda x: 0 if x=="male" else 1 )
```

Código-fonte 34 – Proposta de solução para o desafio de regressão – Parte 1/3
Fonte: Elaborado pelo autor (2023)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Agora chegamos com a novidade! Criamos 2 colunas novas, sendo uma o resultado da multiplicação das alturas dos pais e outra a soma dos logs das alturas dos pais.

```
# Tratamento de dados - CRIAR UMA COLUNA NOVA COM A MULTIPLICAÇÃO DA
ALTURA DOS PAIS
df["multiplicacao"] = df["father"] * df["mother"]

# Tratamento de dados - CRIAR UMA COLUNA NOVA COM A SOMA DO LOG DA
ALTURA DOS PAIS
import math
for col in ["father", "mother"]:
  df[col] = df[col].apply( math.log )
df["soma_log"] = df["father"] + df["mother"]
```

Código-fonte 35 – Proposta de solução para o desafio de regressão – Parte 2/3
Fonte: Elaborado pelo autor (2023)

Por fim, seguimos o pipeline de aprendizado de máquina separando os dados em treino e teste, realizando a predição e aferindo os resultados.

```
# Separação de DADOS e LABELS
X = df.drop(columns=["childHeight", "father", "mother",
"midparentHeight"])
y = df["childHeight"]

# Separação de TREINO e TESTE
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size=0.3,
                                                    random_state=42)

# Padronizar os dados
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Treinar o modelo usando os dados de TREINO
reta = LinearRegression()
reta.fit(X_train, y_train)
y_pred = reta.predict(X_test)

# Avaliar o modelo nos dados de TESTE
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
print(f"MSE = {mse}")
print(f"MAE = {mae}")
print(f"R² = {r2}")
```

Código-fonte 36 – Proposta de solução para o desafio de regressão – Parte 3/3
Fonte: Elaborado pelo autor (2023)

A proposta de solução leva a resultados um pouco melhores do que os apresentados anteriormente, mostrando que a construção de features é uma arte! Parte do trabalho do Cientista de Dados é encontrar e produzir boas features para os modelos de machine learning.

```
MSE = 30.222801760533045
MAE = 4.328557819245586
R² = 0.6557176857583067
```

Figura 31 – Resultado do código-fonte 36
Fonte: Elaborado pelo autor (2023)

CLASSIFICAÇÃO:

A célula abaixo apenas repete os passos iniciais do código de classificação, lendo os dados, e montando o DataFrame para trabalharmos mais facilmente.

```
# Imports necessários
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
from sklearn import datasets

import matplotlib.pyplot as plt

import pandas as pd
import numpy as np

# Obtendo o dataset
tabela = datasets.load_wine()
df = pd.DataFrame(tabela.data, columns=tabela.feature_names)
df["label"] = tabela.target
```

Código-fonte 37 – Proposta de solução para o desafio de classificação – Parte 1/4
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A novidade está agora, quando na célula abaixo traçamos a matriz de correlação entre as features. Essa matriz mostra a relação entre as variáveis: quanto mais próximo de 1, maior é a relação entre elas; quanto mais próximo de 0, menor é a relação entre elas. A relação pode ser positiva (quando uma aumenta, a outra aumenta também), ou negativa (quando o valor de uma variável aumenta, a outra diminui ou vice-versa).

```
# Matriz de correlação de features
import seaborn as sns
sns.heatmap(df.corr());
```

Código-fonte 38 – Proposta de solução para o desafio de classificação – Parte 2/4
Fonte: Elaborado pelo autor (2023)

Note que a matriz também traça as correlações das features com o label. Então, vamos utilizar essa informação para selecionarmos apenas algumas features mais correlacionadas com a variável resposta:

```
# Filtrando apenas as colunas com maior correlação com o label
colunas_para_manter = ["total_phenols", "flavanoids",
                       "od280/od315_of_diluted_wines",
                       "hue", "proline", "label"]
df = df[colunas_para_manter]
```

Código-fonte 39 – Proposta de solução para o desafio de classificação – Parte 3/4
Fonte: Elaborado pelo autor (2023)

E agora seguiremos com o resto padrão da modelagem:

```
# Separar FEATURES do LABEL
X = df.drop(columns=["label"])
y = df["label"]

# Dividir o dataset em TREINO e TESTE
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# Normalizar os dados
mm = MinMaxScaler()
X_train_norm = mm.fit_transform(X_train)
X_test_norm = mm.transform(X_test)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Avaliando o KNN com diferentes valores de K, a partir dos dados
NORMALIZADOS
meu_knn(X_train_norm, X_test_norm, y_train, y_test, 5) # k=5
meu_knn(X_train_norm, X_test_norm, y_train, y_test, 7) # k=7
meu_knn(X_train_norm, X_test_norm, y_train, y_test, 9) # k=9
meu_knn(X_train_norm, X_test_norm, y_train, y_test)    # k=determinado
automaticamente
```

Código-fonte 40 – Proposta de solução para o desafio de classificação – Parte 4/4
Fonte: Elaborado pelo autor (2023)

A proposta de solução leva a resultados um pouco piores do que os apresentados anteriormente. A vantagem é que diminuímos o número de features necessárias para o modelo funcionar. Isso facilita sua utilização na vida real, pois precisaríamos coletar menos informações para poder prever o tipo de vinho.

Outra vantagem é que, no caso do KNN, esse modelo reduzido produziu resultados mais consistentes: note que mesmo variando o valor de K, a métrica permanece a mesma.

```
k=5     Acurácia: 0.9444
k=7     Acurácia: 0.9444
k=9     Acurácia: 0.9444
k=13    Acurácia: 0.9444
```

Figura 32 – Resultado do código-fonte 40
Fonte: Elaborado pelo autor (2023)

CLUSTERIZAÇÃO:

Para o desafio de clusterização, vamos assumir que você já possui o código executado para geração de 5 clusteres a partir dos dados de entrada. Agora, vamos criar uma coluna no DataFrame para armazenar a qual cluster cada exemplo foi atribuído. Fazemos isso facilmente em Pandas:

```
# Criando nova coluna para armezenar os clusteres
dados["cluster"] = kmeans_labels
```

Código-fonte 41 – Proposta de solução para o desafio de clusterização – Parte 1/2
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

E dessa forma, podemos filtrar os dados de um cluster por vez, analisando seus dados principais para traçar o perfil daquele cluster. Por exemplo, veja como filtrar as pessoas do cluster 0:

```
# Traçando o perfil do cluster 0
dados[ dados["cluster"] == 0 ].describe()
```

Código-fonte 42 – Proposta de solução para o desafio de clusterização – Parte 2/2
Fonte: Elaborado pelo autor (2023)

O código-fonte 42 produz a saída da Figura 32. Ela revela que o cluster zero é formado por pessoas idade média de 41 anos, alto rendimento anual e gastos baixos.

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)	cluster
count	35.000000	35.000000	35.000000	35.000000	35.0
mean	164.371429	41.114286	88.200000	17.114286	0.0
std	21.457325	11.341676	16.399067	9.952154	0.0
min	125.000000	19.000000	70.000000	1.000000	0.0
25%	148.000000	34.000000	77.500000	10.000000	0.0
50%	165.000000	42.000000	85.000000	16.000000	0.0
75%	182.000000	47.500000	97.500000	23.500000	0.0
max	199.000000	59.000000	137.000000	39.000000	0.0

Figura 33 – Resultado do código-fonte 42
Fonte: Elaborado pelo autor (2023)

Por outro lado, ao alterar o código-fonte 42 para selecionar pessoas do cluster 4, teríamos a saída da Figura 33. Ela revela que o cluster quatro é formado por pessoas idade média de 45 anos, baixo rendimento anual e gastos baixos.

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)	cluster
count	23.00000	23.000000	23.000000	23.000000	23.0
mean	23.00000	45.217391	26.304348	20.913043	4.0
std	13.56466	13.228607	7.893811	13.017167	0.0
min	1.00000	19.000000	15.000000	3.000000	4.0
25%	12.00000	35.500000	19.500000	9.500000	4.0
50%	23.00000	46.000000	25.000000	17.000000	4.0
75%	34.00000	53.500000	33.000000	33.500000	4.0
max	45.00000	67.000000	39.000000	40.000000	4.0

Figura 34 – Resultado do código-fonte 42, alterando o ID de cluster para 4

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Fonte: Elaborado pelo autor (2023)

Seguindo esse processo, você pode analisar cada um dos clusteres formados e entender os perfis de consumidores de cada um deles. Incluindo idades limítrofes, desvio padrão de rendimento e outras análises pertinentes.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

IZBICKI, Rafael; DOS SANTOS, Tiago Mendonça. **Aprendizado de máquina:** uma abordagem estatística. Rafael Izbicki, 2020.

PROVOST, Foster. Data Science para Negócios by Tom Fawcett Foster Provost [Fawcett, Tom](z-lib. org).

RUSSELL, Stuart J.; NORVIG, Peter. **Inteligencia Artificial:** un enfoque moderno. 2004.

SURESH, Harini; GUTTAG, John. **A framework for understanding sources of harm throughout the machine learning life cycle**. In: Equity and access in algorithms, mechanisms, and optimization. 2021. p. 1-9.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

GLOSSÁRIO

Termo	**Explicação**
ADALINE	Adaptive Linear Neuron
CNN	Convolutional Neural Network
IA	Inteligência Artificial
IoT	Internet das Coisas
NLP	Natural Language Processing
PLN	Processamento de Linguagem Natural
RNN	Recurrent Neural Network

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com